Отчёт по лабораторной работе №2

Имитационное моделирование

Бахарев Дмитрий Александрович

Содержание

1 Цель работы **7**

2 Задание **8**

3 Теоретическое введение **9**
3.1 Модель SIR 9
3.2 Модель Лотки-Вольтерра 9

4 Выполнение лабораторной работы **11**
4.1 Создание проекта DrWatson 11
4.2 Запуск базового скрипта SIR 11
4.3 Генерация производных форматов 12
4.4 Код скрипта sir_ode.jl 13
4.5 Результаты SIR: основная динамика 14
4.6 Фазовый портрет SIR 15
4.7 Результаты LV: спектр и фазовый портрет 16
4.8 Параметрическое исследование SIR 17
4.9 Параметрическое исследование LV 18
4.10 Jupyter-ноутбуки 20
4.11 Листинги из Julia 21

5 Модель SIR **22**
5.1 Активация проекта и загрузка пакетов 22
5.2 Определение модели SIR 23
5.3 Параметры и решение 24
5.4 Подготовка данных 25
5.5 1. Основной график: динамика всех трёх групп 28
5.6 2. График только инфицированных (I) 29
5.7 3. Логарифмический масштаб (для анализа экспоненциального роста) 30
5.8 4. Проценты популяции 31
5.9 5. Фазовый портрет S–I 32
5.10 6. Эффективное репродуктивное число R_e(t) 33
5.11 7. Компактная панель всех кривых 34
5.12 Сохранение графиков 35
5.13 Бенчмарк производительности 36
5.14 Дополнительный анализ 36

5.15 Сохранение результатов 38

6 Модель Лотки-Вольтерры (хищник-жертва) 39
6.1 Активация проекта и загрузка пакетов 39
6.2 Определение модели 40
6.3 Параметры и начальные условия 41
6.4 Подготовка данных 43
6.5 Вывод информации о модели 47
6.6 Стационарные точки 48
6.7 График 1: динамика популяций 48
6.8 График 2: фазовый портрет с изоклинами и стрелками 49
6.9 График 3: производные популяций 53
6.10 График 4: относительные изменения (%) 54
6.11 График 5: спектральный анализ (Фурье) 55
6.12 График 6: компактная панель всех графиков 58
6.13 Сохранение графиков 60
6.14 Анализ результатов 61
6.15 Сохранение результатов 63

7 Параметрическое исследование модели SIR 64
7.1 Активация проекта и загрузка пакетов 64
7.2 Определение модели SIR 65
7.3 Базовые параметры эксперимента 65
7.4 Функция запуска одного эксперимента 66
7.5 Запуск базового эксперимента (одна комбинация) 67
7.6 Визуализация базового эксперимента 69
7.7 Параметрическое сканирование по β и γ 69
7.8 Запуск всех экспериментов и сбор сводных результатов 70
7.9 Сводная таблица 72
7.10 Сравнительный график всех траекторий I(t) 72
7.11 График R0 от параметров 73
7.12 График R(∞) от параметров 74
7.13 Бенчмаркинг для разных значений β 74
7.14 Сохранение всех результатов 76

8 Параметрическое исследование модели Лотки-Вольтерры 78
8.1 Активация проекта и загрузка пакетов 78
8.2 Определение модели Лотки-Вольтерры 79
8.3 Базовые параметры эксперимента 79
8.4 Функция запуска одного эксперимента 80
8.5 Запуск базового эксперимента 82
8.6 Визуализация базового эксперимента 83
8.7 Параметрическое сканирование по α и γ 83

8.8 Запуск всех экспериментов и сбор сводных результатов 84
8.9 Сводная таблица 86
8.10 Сравнительный график жертв 87
8.11 Сравнительный график хищников 87
8.12 Фазовые портреты для всех комбинаций 88
8.13 Спектральный анализ — период колебаний 89
8.14 Бенчмаркинг 91
8.15 Сохранение всех результатов 93

9 Заключение **94**

Список литературы **95**

Список иллюстраций

4.1 Создание проекта DrWatson 11
4.2 Запуск базового скрипта SIR 12
4.3 Генерация форматов для sir_ode.jl и lv_ode.jl 13
4.4 Генерация форматов для параметрических скриптов 13
4.5 Код скрипта sir_ode.jl 14
4.6 Динамика групп S, I, R 14
4.7 Панель графиков SIR 15
4.8 Фазовый портрет SIR 16
4.9 Спектр модели Лотки-Вольтерра 16
4.10 Фазовый портрет Лотки-Вольтерра 17
4.11 Параметрическое исследование SIR 18
4.12 Зависимость R_0 от параметров 18
4.13 Параметрическое исследование жертв 19
4.14 Параметрическое исследование хищников 19
4.15 Зависимость периода от параметров 20
4.16 Выполнение ноутбука SIR 20
4.17 Выполнение ноутбука LV 21

Список таблиц

1 Цель работы

Изучить базовые модели имитационного моделирования (SIR и Лотки-Вольтерра), освоить их численную реализацию в Julia и провести параметрические исследования.

2 Задание

1. Реализовать модель эпидемии **SIR** в Julia.
2. Реализовать модель **«хищник-жертва»** Лотки-Вольтерра.
3. Провести параметрические исследования для обеих моделей.
4. Построить графики и проанализировать результаты.
5. Сгенерировать отчёт в форматах PDF и DOCX.

3 Теоретическое введение

3.1 Модель SIR

Модель SIR описывает распространение инфекции в популяции. Популяция делится на три группы:

- $S(t)$ — восприимчивые к болезни, но пока здоровые;
- $I(t)$ — инфицированные, распространяющие инфекцию;
- $R(t)$ — выздоровевшие с иммунитетом.

Система уравнений:

$$\begin{cases} \dfrac{dS}{dt} = -\beta c \dfrac{SI}{N}, \\ \dfrac{dI}{dt} = \beta c \dfrac{SI}{N} - \gamma I, \\ \dfrac{dR}{dt} = \gamma I. \end{cases}$$

Здесь β — вероятность заражения при контакте, c — среднее число контактов, γ — скорость выздоровления. Базовое репродуктивное число: $R_0 = c\beta/\gamma$.

3.2 Модель Лотки-Вольтерра

Модель «хищник-жертва» описывает взаимодействие двух популяций:

$$\begin{cases} \dfrac{dx}{dt} = \alpha x - \beta xy, \\ \dfrac{dy}{dt} = -\gamma y + \delta xy. \end{cases}$$

Здесь x — численность жертв, y — численность хищников. Стационарное состояние: $x^* = \gamma/\delta$, $y^* = \alpha/\beta$.

4 Выполнение лабораторной работы

4.1 Создание проекта DrWatson

Проект DrWatson создан в каталоге `labs/lab02/project` командой `initialize_project` (рис. Рисунок 4.1).

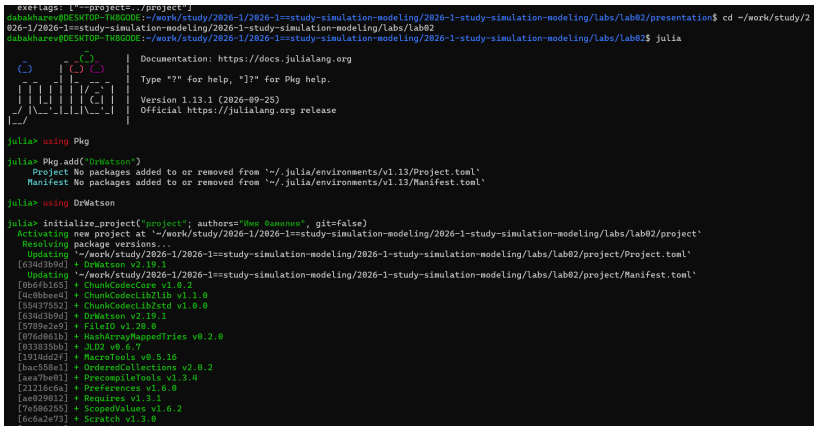


Рисунок 4.1: Создание проекта DrWatson

4.2 Запуск базового скрипта SIR

Скрипт `sir_ode.jl` реализует модель SIR. Параметры модели:

Параметр	Значение
β (вероятность заражения)	0.05
c (среднее число контактов)	10.0
γ (скорость выздоровления)	0.25
$R_0 = c \cdot \beta / \gamma$	**2.0**
Средняя продолжительность болезни	4 дня

Начальные условия: $S_0 = 990$, $I_0 = 10$, $R_0 = 0$.

Результаты численного решения приведены на рис. Рисунок 4.2.

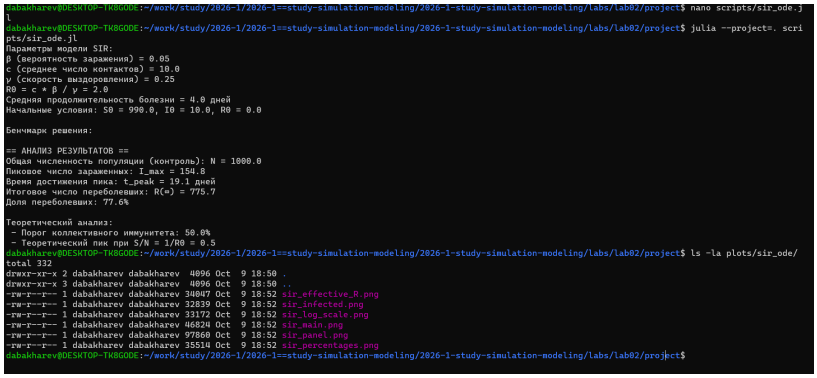


Рисунок 4.2: Запуск базового скрипта SIR

- Общая численность: $N = 1000$;
- Пиковое число заражённых: $I_{max} = 154.8$;
- Время достижения пика: $t_{peak} = 19.1$ дней;
- Итоговое число переболевших: $R(\infty) = 775.7$ (77.6%);
- Порог коллективного иммунитета: 50%.

4.3 Генерация производных форматов

Через `tangle.jl` для обоих базовых скриптов сгенерированы три формата: чистый скрипт, документация Quarto и Jupyter Notebook (рис. Рисунок 4.3).



Рисунок 4.3: Генерация форматов для sir_ode.jl и lv_ode.jl

Для параметрических скриптов также сгенерированы три формата (рис. Рисунок 4.4).

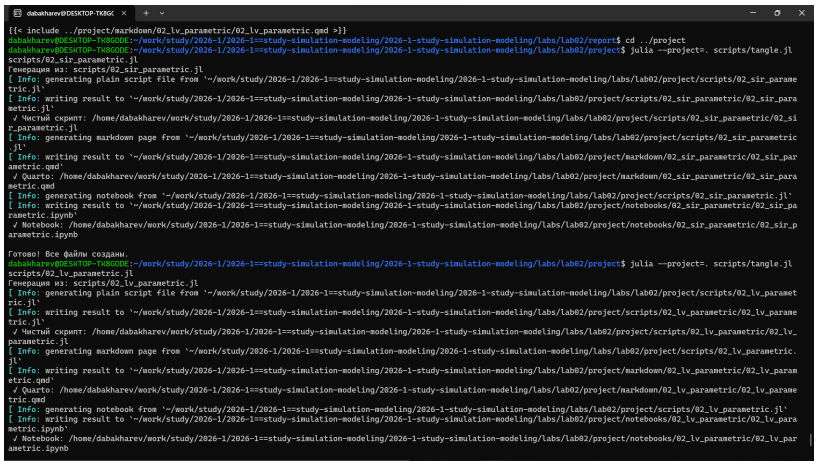


Рисунок 4.4: Генерация форматов для параметрических скриптов

4.4 Код скрипта sir_ode.jl

Код базового скрипта модели SIR приведён на рис. Рисунок 4.5.

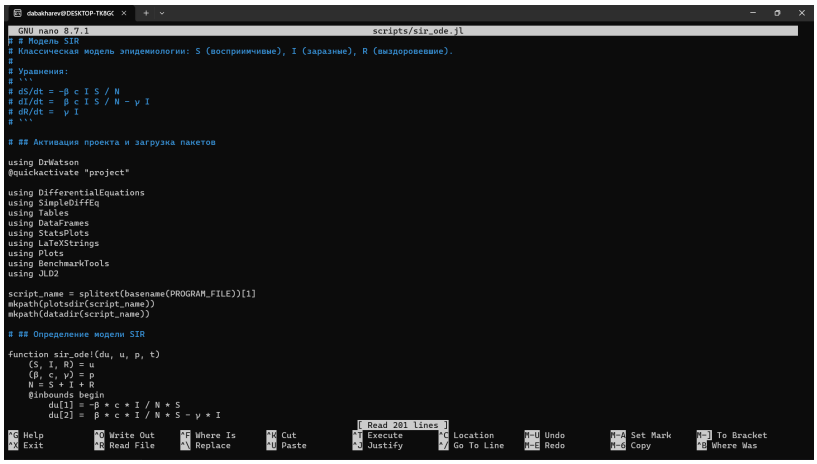


Рисунок 4.5: Код скрипта sir_ode.jl

4.5 Результаты SIR: основная динамика

Динамика трёх групп S(t), I(t), R(t) приведена на рис. Рисунок 4.6.

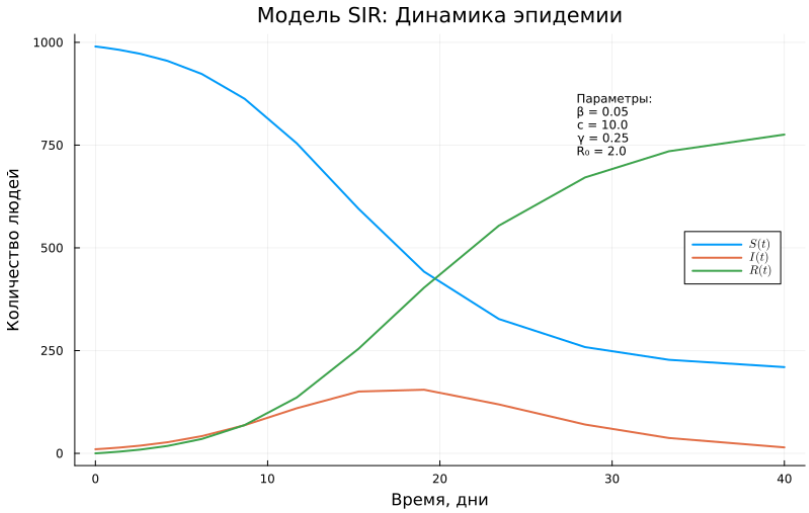


Рисунок 4.6: Динамика групп S, I, R

Панель с несколькими графиками SIR показана на рис. Рисунок 4.7.

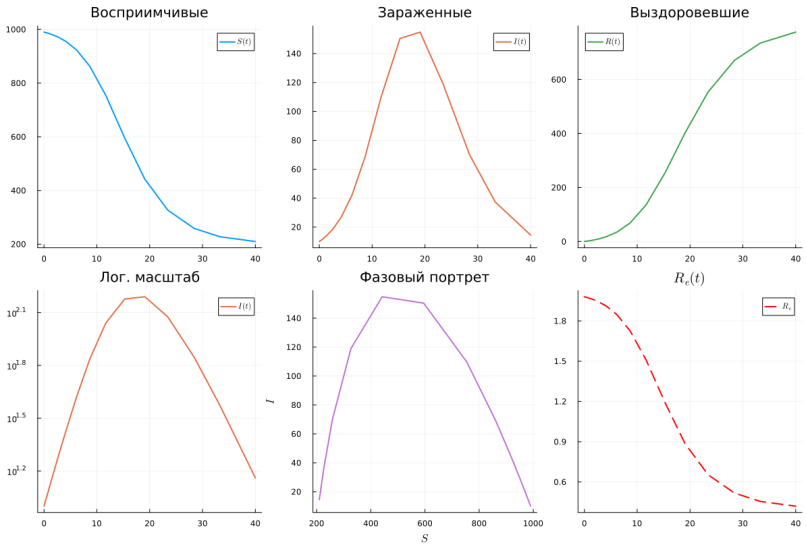


Рисунок 4.7: Панель графиков SIR

Из графиков видно:

- Число восприимчивых $S(t)$ монотонно убывает — люди переболевают;
- Число инфицированных $I(t)$ имеет **характерный пик** на 19-й день;
- Число переболевших $R(t)$ растёт и выходит на плато ≈ 776.

4.6 Фазовый портрет SIR

Фазовый портрет модели SIR показан на рис. Рисунок 4.8.

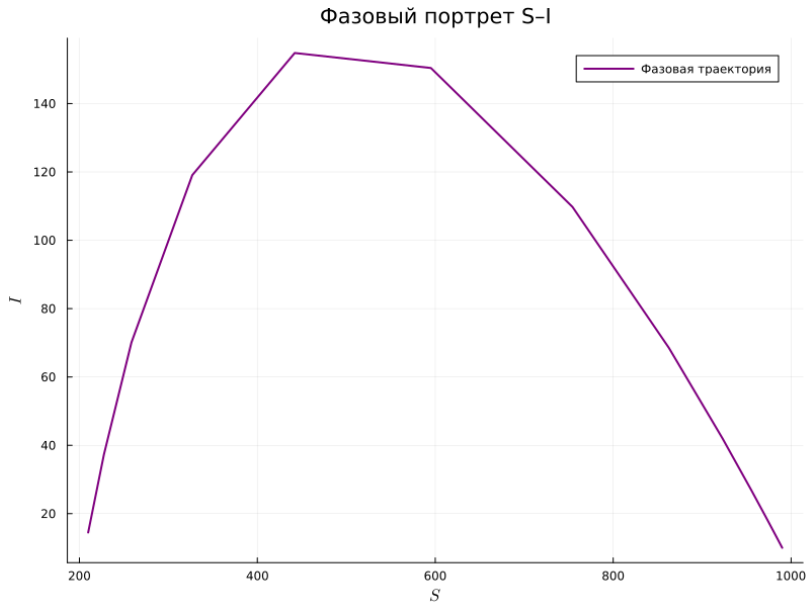


Рисунок 4.8: Фазовый портрет SIR

4.7 Результаты LV: спектр и фазовый портрет

Спектр модели Лотки-Вольтерра приведён на рис. Рисунок 4.9.

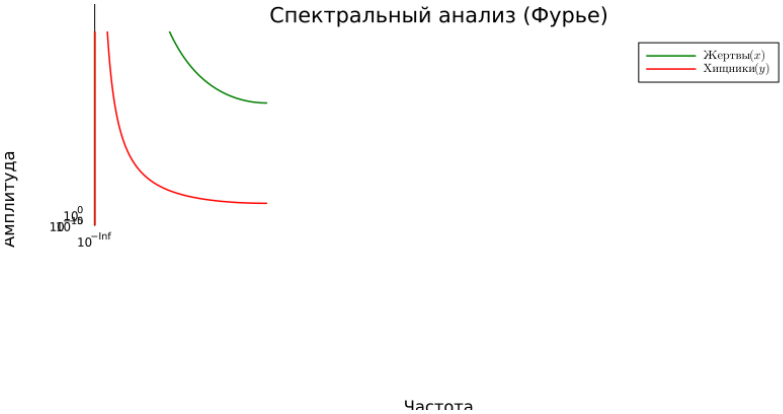


Рисунок 4.9: Спектр модели Лотки-Вольтерра

Фазовый портрет модели LV показан на рис. Рисунок 4.10.

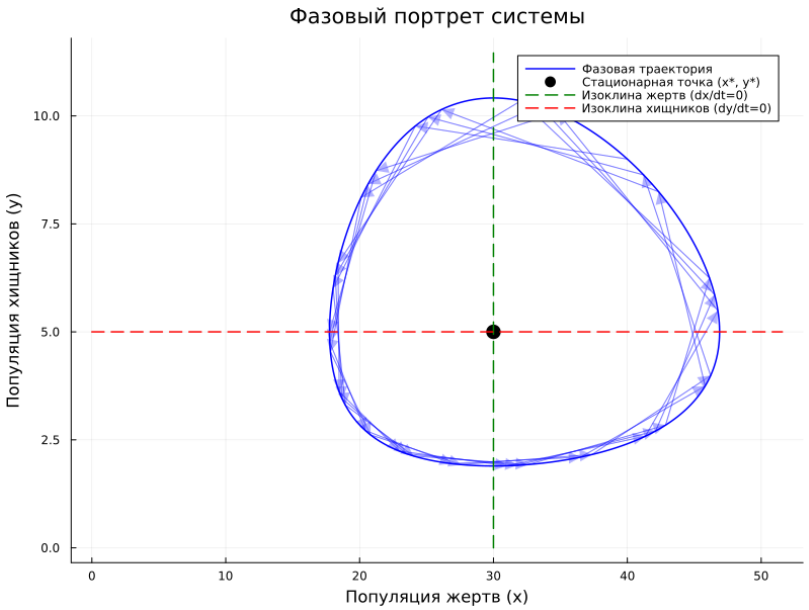


Рисунок 4.10: Фазовый портрет Лотки-Вольтерра

Из графиков видно:

- Динамика жертв и хищников имеет **колебательный характер**;
- Колебания происходят **в противофазе**;
- Фазовый портрет — **замкнутая кривая** (консервативная система).

4.8 Параметрическое исследование SIR

Сравнение траекторий при разных β приведено на рис. Рисунок 4.11.

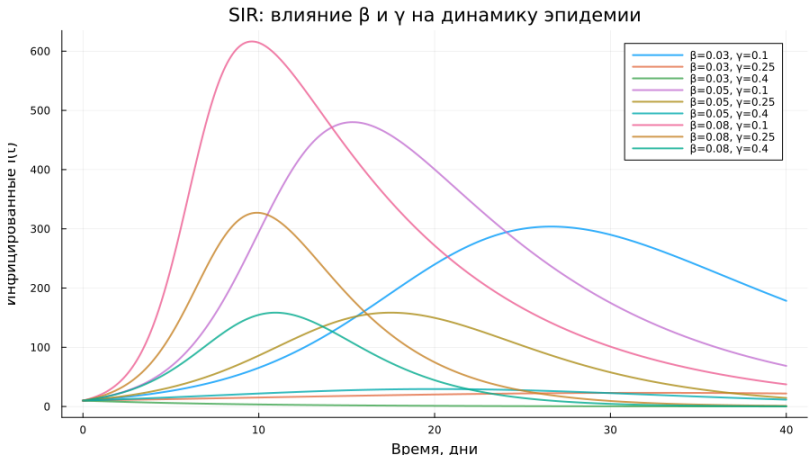


Рисунок 4.11: Параметрическое исследование SIR

Зависимость R_0 от параметров показана на рис. Рисунок 4.12.

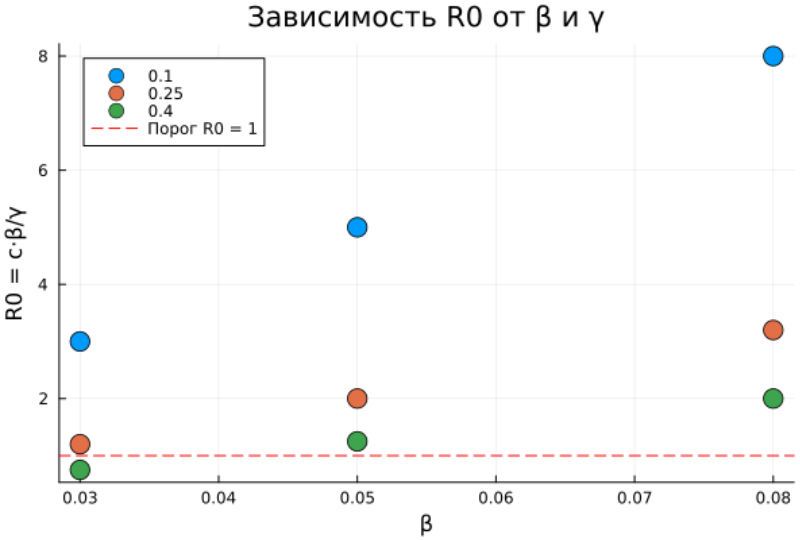


Рисунок 4.12: Зависимость R_0 от параметров

4.9 Параметрическое исследование LV

Параметрическое исследование динамики жертв приведено на рис. Рисунок 4.13.

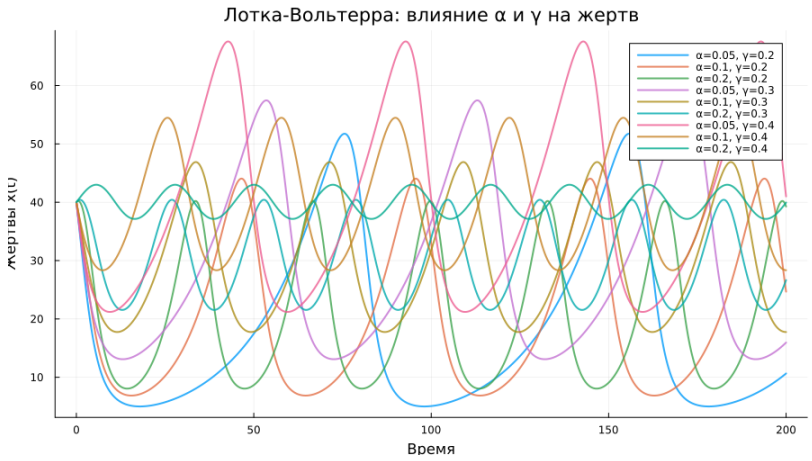


Рисунок 4.13: Параметрическое исследование жертв

Параметрическое исследование динамики хищников показано на рис. Рисунок 4.14.

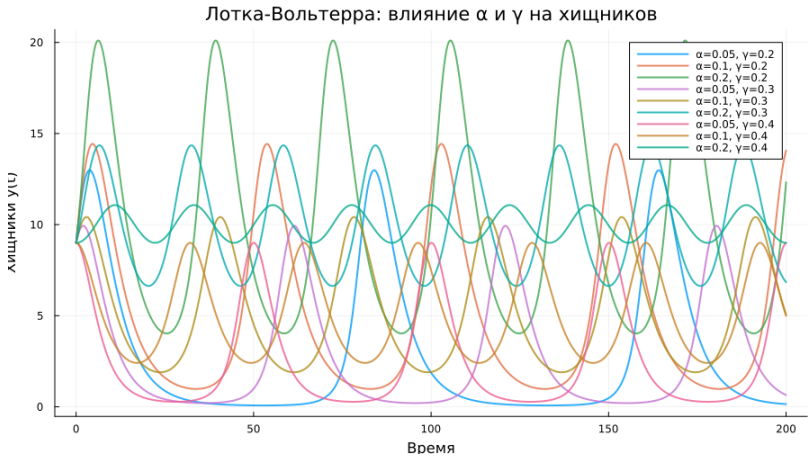


Рисунок 4.14: Параметрическое исследование хищников

Зависимость периода колебаний от параметров приведена на рис. Рисунок 4.15.

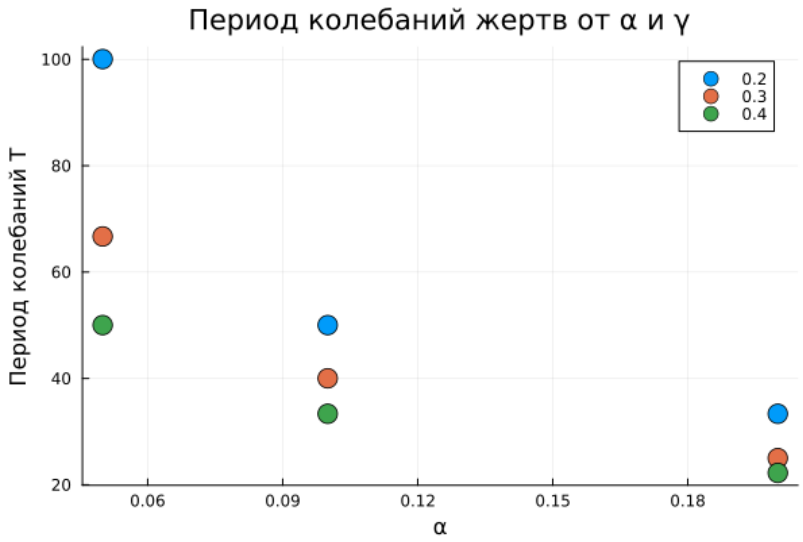


Рисунок 4.15: Зависимость периода от параметров

4.10 Jupyter-ноутбуки

Jupyter-ноутбук `sir_ode.ipynb` выполнен (рис. Рисунок 4.16).

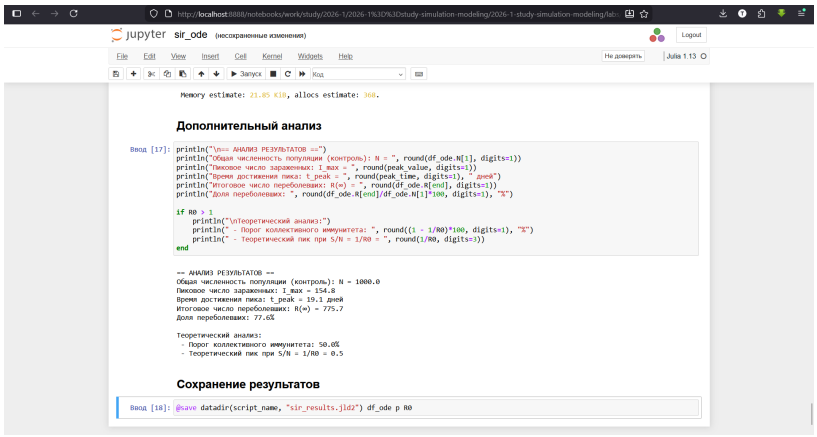


Рисунок 4.16: Выполнение ноутбука SIR

Jupyter-ноутбук `lv_ode.ipynb` выполнен (рис. Рисунок 4.17).

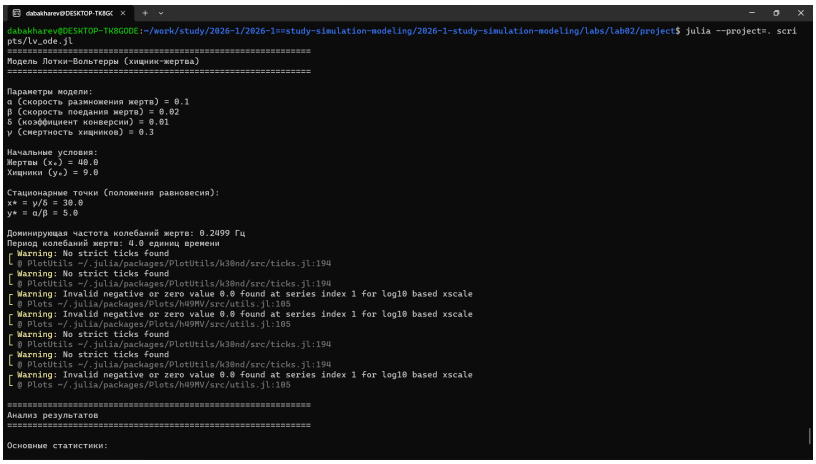


Рисунок 4.17: Выполнение ноутбука LV

4.11 Листинги из Julia

5 Модель SIR

Классическая модель эпидемиологии: S (восприимчивые), I (заразные), R (выздоровевшие).

Уравнения:

```
dS/dt = -β c I S / N
dI/dt =  β c I S / N - γ I
dR/dt =  γ I
```

5.1 Активация проекта и загрузка пакетов

```
import Pkg
Pkg.activate("/home/dabakharev/work/study/2026-1/2026-1==study-simu
using DrWatson



using DifferentialEquations
using SimpleDiffEq
using Tables
using DataFrames
using StatsPlots
using LaTeXStrings
```

```
using Plots
ENV["GKSwstype"] = "100"
gr(format=:png)
using BenchmarkTools
using JLD2

script_name = splitext(basename(PROGRAM_FILE))[1]
mkpath(plotsdir(script_name))
mkpath(datadir(script_name))
```

```
  Activating project at `~/work/study/2026-1/2026-1==study-simulation
```

```
"/home/dabakharev/work/study/2026-1/2026-1==study-simulation-modeli
```

5.2 Определение модели SIR

```
function sir_ode!(du, u, p, t)
    (S, I, R) = u
    (β, c, γ) = p
    N = S + I + R
    @inbounds begin
        du[1] = -β * c * I / N * S
        du[2] =  β * c * I / N * S - γ * I
        du[3] =  γ * I
    end
    nothing
end
```

```
sir_ode! (generic function with 1 method)
```

5.3 Параметры и решение

```
□t = 0.1
tmax = 40.0
tspan = (0.0, tmax)
u0 = [990.0, 10.0, 0.0]    # S, I, R
p = [0.05, 10.0, 0.25]     # □, c, □

R0 = (p[2] * p[1]) / p[3]  # R0 = c·□ / □

prob_ode = ODEProblem(sir_ode!, u0, tspan, p)
sol_ode = solve(prob_ode, dt=□t)
```

```
retcode: Success
Interpolation: 3rd order Hermite
t: 15-element Vector{Float64}:
  0.0
  0.1
  0.5331242537191838
  1.3919008491040001
  2.6102120755226386
  4.1811820758323055
  6.180123810566044
  8.664951930903612
 11.691425331813486
 15.279370667110317
 19.082963490744028
 23.423425193442526
 28.429493730986167
```

```
 33.29877917363959
 40.0
u: 15-element Vector{Vector{Float64}}:
 [990.0, 10.0, 0.0]
 [989.4990153200853, 10.247898072276143, 0.25308660763853125]
 [987.1852964117763, 11.391111426852982, 1.423592161370628]
 [981.8312125227972, 14.026022042698541, 4.142765434504279]
 [972.1392310589528, 18.75781556165418, 9.10295337939301]
 [954.9905500491583, 27.007702518267603, 18.00174743257403]
 [923.0706717929863, 41.92976779832725, 34.9995604086864]
 [862.6705810342163, 68.49345419063431, 68.83596477514939]
 [754.3022993851439, 109.741861854093, 135.95583876076313]
 [595.1014268847732, 150.41363264480688, 254.4849404704199]
 [442.0141179101298, 154.80694988491848, 403.17893220495165]
 [326.8951167493958, 119.06860394684853, 554.0362793037556]
 [258.50852421889664, 70.11052194753057, 671.3809538335727]
 [227.58338337085735, 37.32857063157991, 735.0880459975626]
 [209.8412140201437, 14.473644966455723, 775.6851410134004]
```

5.4 Подготовка данных

```
df_ode = DataFrame(Tables.table(sol_ode'))
rename!(df_ode, ["S", "I", "R"])
df_ode[!, :t] = sol_ode.t
df_ode[!, :N] = df_ode.S + df_ode.I + df_ode.R
```

```
15-element Vector{Float64}:
 1000.0
```

```
 1000.0
  999.9999999999999
 1000.0
 1000.0
  999.9999999999999
 1000.0
 1000.0
 1000.0
 1000.0
 1000.0
  999.9999999999999
  999.9999999999999
  999.9999999999999
  999.9999999999998
```

Вычисляем R_e(t) = R0 · S(t)/N — эффективное репродуктивное число

```
df_ode[!, :Re] = R0 .* df_ode.S ./ df_ode.N
```

```
15-element Vector{Float64}:
 1.98
 1.9789980306401707
 1.9743705928235529
 1.9636624250455943
 1.9442784621179054
 1.909981100098317
 1.8461413435859726
 1.7253411620684327
 1.5086045987702879
 1.1902028537695464
```

```
 0.8840282358202596
 0.6537902334987917
 0.5170170484377934
 0.45516676674171475
 0.4196824280402875
```

Доли в процентах

```
df_ode[!, :S_pct] = df_ode.S ./ df_ode.N .* 100
df_ode[!, :I_pct] = df_ode.I ./ df_ode.N .* 100
df_ode[!, :R_pct] = df_ode.R ./ df_ode.N .* 100
```

```
15-element Vector{Float64}:
  0.0
  0.025308660763853127
  0.1423592161370628
  0.41427654345042786
  0.9102953379393011
  1.8001747432574033
  3.4999560408686397
  6.8835964775149385
 13.595583876076315
 25.44849404704199
 40.317893220495165
 55.40362793037556
 67.13809538335728
 73.50880459975627
 77.56851410134006
```

Вывод параметров модели

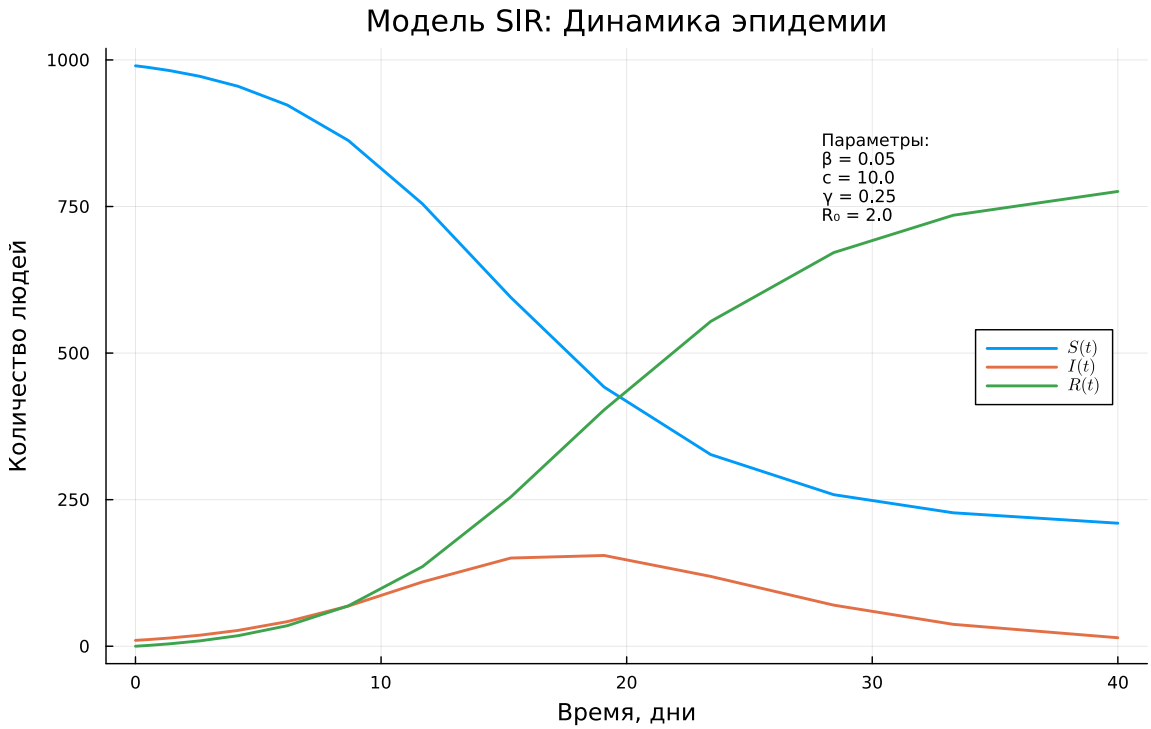
```
println("□□□□□□□□□ □□□□□□ SIR:")
println("□ (□□□□□□□□□□ □□□□□□□□□) = ", p[1])
println("c (□□□□□□□ □□□□□ □□□□□□□□□) = ", p[2])
println("□ (□□□□□□□□ □□□□□□□□□□□□□) = ", p[3])
println("R0 = c * □ / □ = ", round(R0, digits=3))
println("□□□□□□□ □□□□□□□□□□□□□□□□□ □□□□□□□ = ", round(1/p[3], digits=2), " □□
println("□□□□□□□□□ □□□□□□□: S0 = ", u0[1], ", I0 = ", u0[2], ", R0 = ", u0[3
```

```
□□□□□□□□□ □□□□□□ SIR:
□ (□□□□□□□□□□ □□□□□□□□□) = 0.05
c (□□□□□□□ □□□□□ □□□□□□□□□) = 10.0
□ (□□□□□□□□ □□□□□□□□□□□□□) = 0.25
R0 = c * □ / □ = 2.0
□□□□□□□ □□□□□□□□□□□□□□□□□ □□□□□□□ = 4.0 □□□□
□□□□□□□□□ □□□□□□□: S0 = 990.0, I0 = 10.0, R0 = 0.0
```

5.5 1. Основной график: динамика всех трёх групп

```
plt1 = @df df_ode plot(:t, [:S :I :R],
    label=[L"S(t)" L"I(t)" L"R(t)"],
    xlabel="□□□□□, □□□", ylabel="□□□□□□□□□□ □□□□□",
    title="□□□□□□ SIR: □□□□□□□□ □□□□□□□□",
   linewidth=2, legend=:right, grid=true, size=(800, 500))

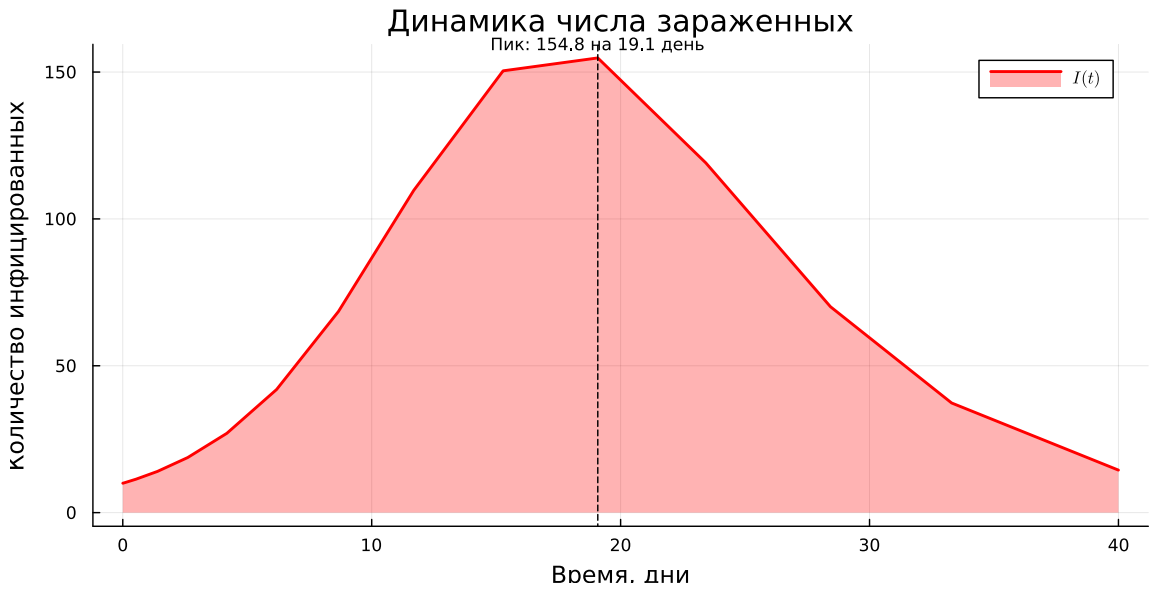
annotate!(plt1,
    maximum(df_ode.t) * 0.7,
    maximum(df_ode.N) * 0.8,
   text("□□□□□□□□□:\n□ = $(p[1])\nc = $(p[2])\n□ = $(p[3])\nR₀ = $(round(
```



5.6 2. График только инфицированных (I)

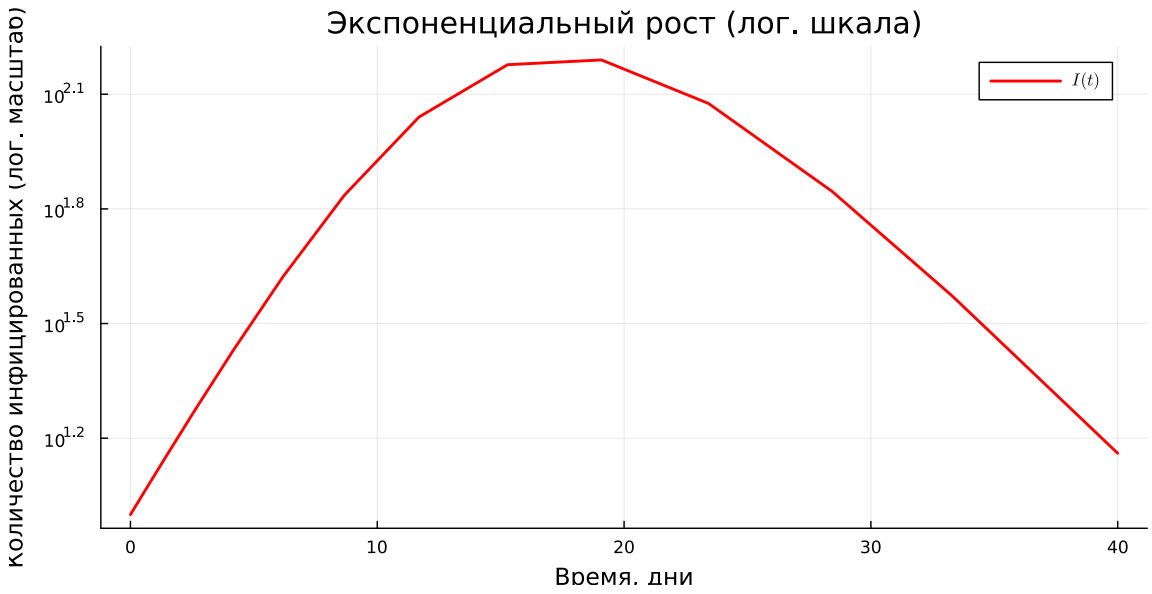
```
plt2 = @df df_ode plot(:t, :I,
   label=L"I(t)", xlabel="□□□□□, □□□", ylabel="□□□□□□□□□□ □□□□□□□□□□□□□□□",
    title="□□□□□□□□ □□□□□ □□□□□□□□□□",
    color=:red, linewidth=2, fill=(0, 0.3, :red),
    grid=true, size=(800, 400))

peak_idx = argmax(df_ode.I)
peak_time = df_ode.t[peak_idx]
peak_value = df_ode.I[peak_idx]
vline!(plt2, [peak_time], color=:black, linestyle=:dash, label=false,
annotate!(plt2, peak_time, peak_value * 1.05,
   text("□□□: $(round(peak_value, digits=1)) □□ $(round(peak_time, digi
```



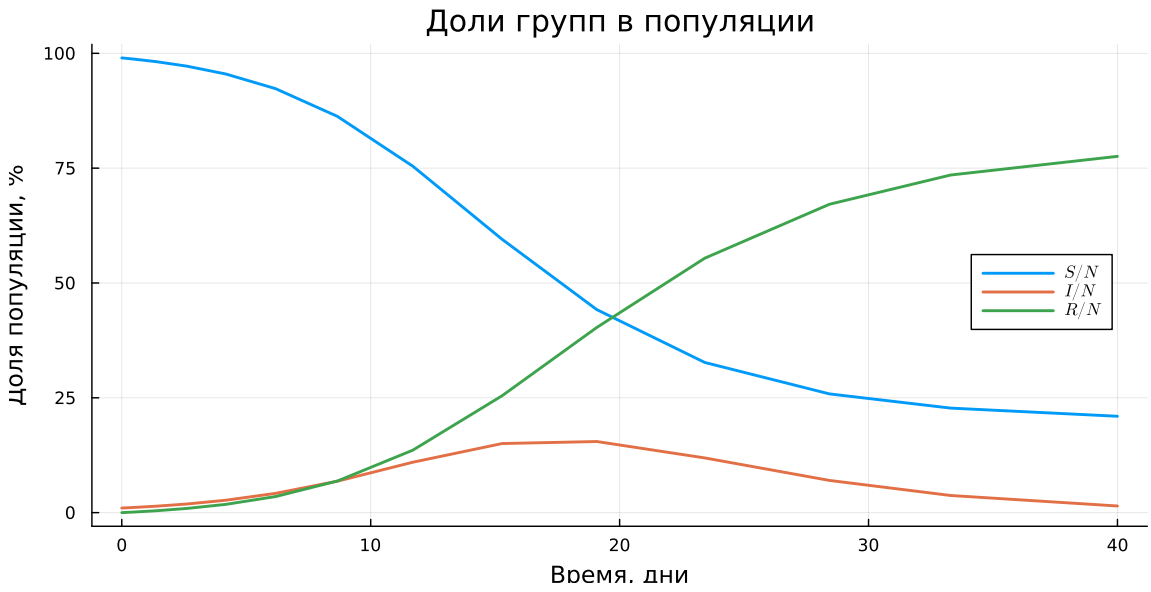
5.7 3. Логарифмический масштаб (для анализа экспоненциального роста)

```
plt3 = @df df_ode plot(:t, :I,
    label=L"I(t)", xlabel="□□□□□, □□□", ylabel="□□□□□□□□□□ □□□□□□□□□□□□□□ (□□□
     title="□□□□□□□□□□□□□□□□ □□□□ (□□□. □□□□□)",
    yscale=:log10, color=:red, linewidth=2, grid=true, size=(800, 400)
```



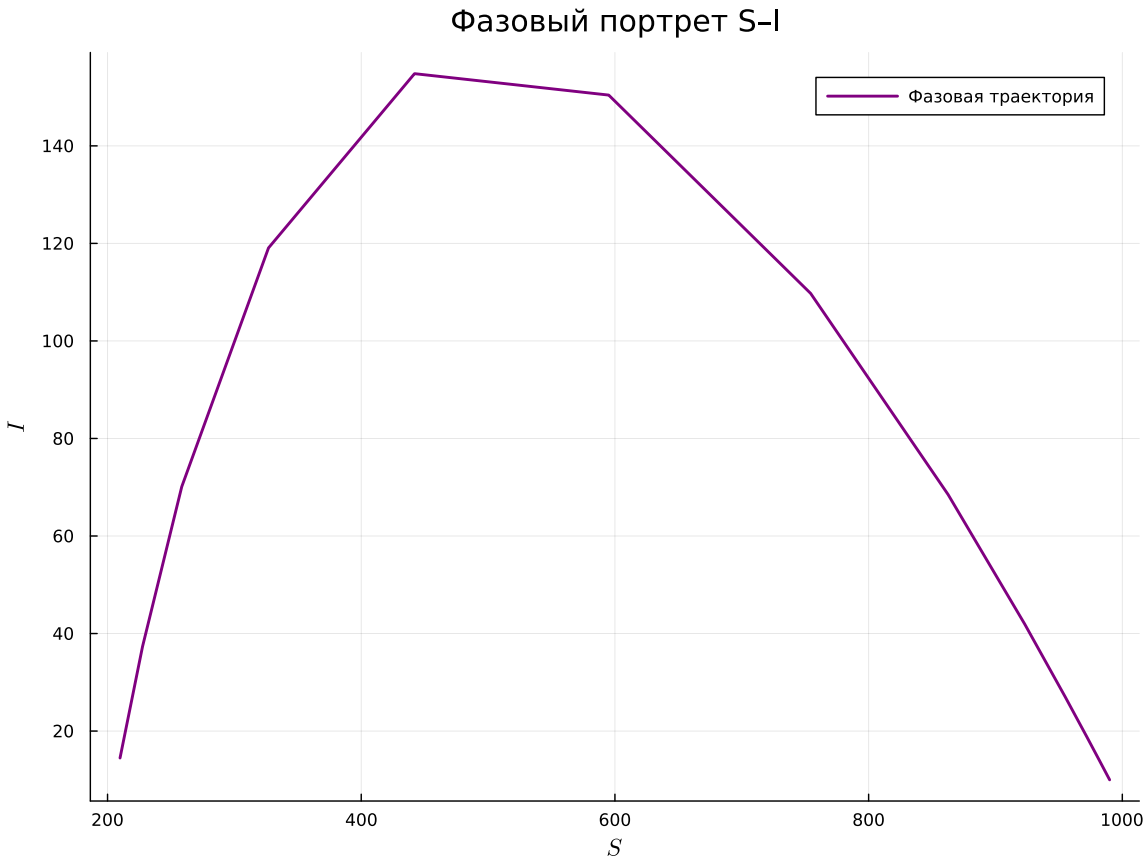
5.8 4. Проценты популяции

```
plt4 = @df df_ode plot(:t, [:S_pct :I_pct :R_pct],
    label=[L"S/N" L"I/N" L"R/N"],
    xlabel="□□□□□, □□□", ylabel="□□□□ □□□□□□□□□, %",
    title="□□□□ □□□□□ □ □□□□□□□□□",
   linewidth=2, legend=:right, grid=true, size=(800, 400))
```



5.9 5. Фазовый портрет S–I

```
plt5 = @df df_ode plot(:S, :I,
    label="□□□□□□□ □□□□□□□□□□",
    xlabel=L"S", ylabel=L"I",
    title="□□□□□□□ □□□□□□□ S-I",
   color=:purple, linewidth=2, grid=true, size=(800, 600))
```



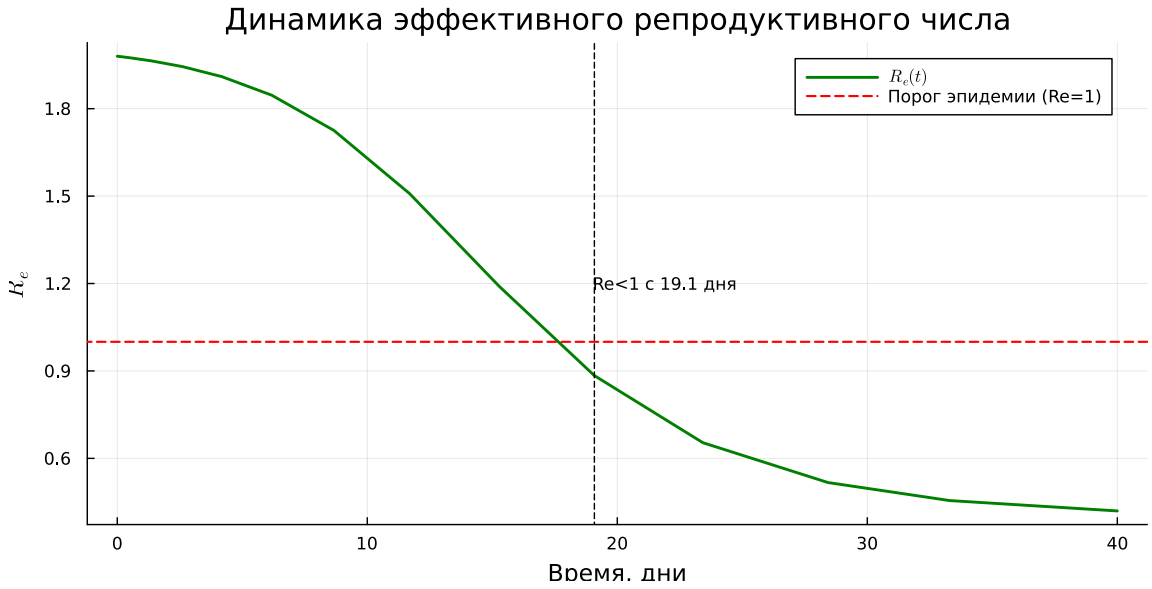
5.10 6. Эффективное репродуктивное число R_e(t)

```
plt6 = @df df_ode plot(:t, :Re,
    label=L"R_e(t)", xlabel="□□□□□, □□□", ylabel=L"R_e",
    title="□□□□□□□□ □□□□□□□□□□□□ □□□□□□□□□□□□□□□□ □□□□□",
   color=:green, linewidth=2, grid=true, size=(800, 400))

hline!(plt6, [1.0], color=:red, linestyle=:dash,
    label="□□□□□ □□□□□□□□ (Re=1)", linewidth=1.5)

cross_idx = findfirst(x -> x < 1, df_ode.Re)
if !isnothing(cross_idx) && cross_idx > 1
```

```
    cross_time = df_ode.t[cross_idx]
    vline!(plt6, [cross_time], color=:black, linestyle=:dash, label=fa
    annotate!(plt6, cross_time, 1.2,
      text("Re<1 □ $(round(cross_time, digits=1)) □□□", 8, :left))
end
```

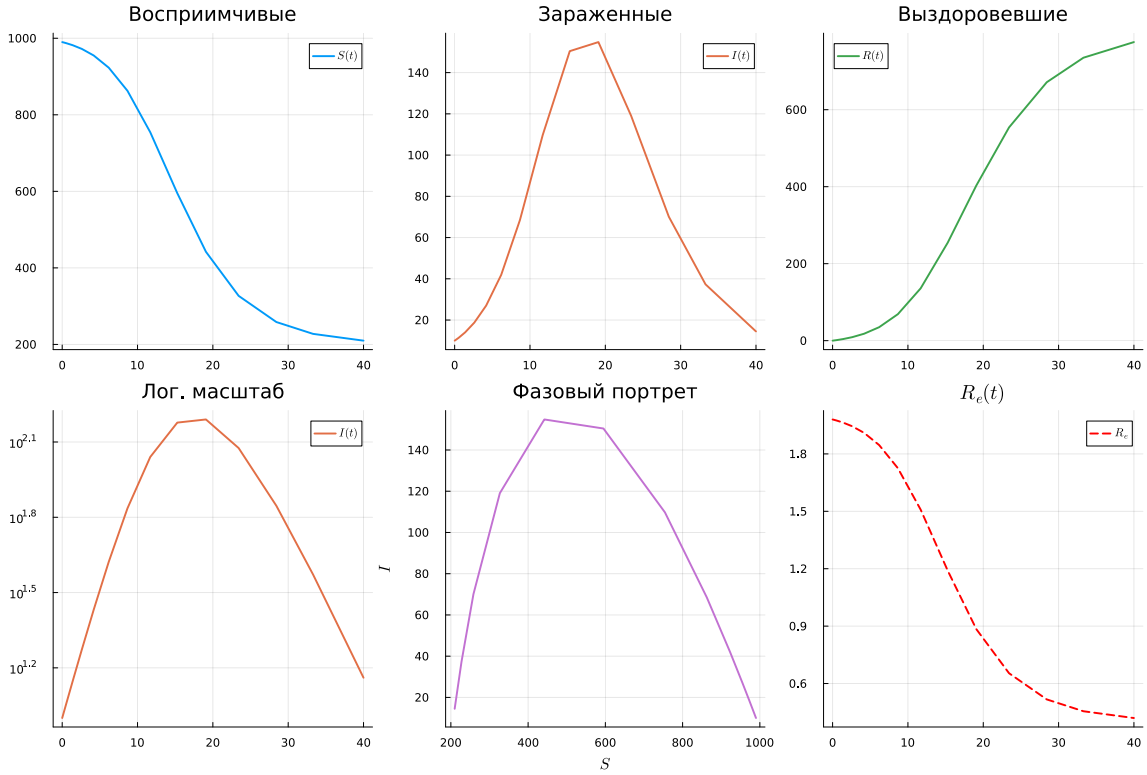


5.11 7. Компактная панель всех кривых

```
plt7 = plot(layout=(2, 3), size=(1200, 800))

plot!(plt7[1], df_ode.t, df_ode.S, label=L"S(t)", color=1,
    linewidth=2, title="□□□□□□□□□□□□□")
plot!(plt7[2], df_ode.t, df_ode.I, label=L"I(t)", color=2,
    linewidth=2, title="□□□□□□□□□")
plot!(plt7[3], df_ode.t, df_ode.R, label=L"R(t)", color=3,
    linewidth=2, title="□□□□□□□□□□□□□")
```

```
plot!(plt7[4], df_ode.t, df_ode.I, label=L"I(t)", color=2,
    linewidth=2, yscale=:log10, title="□□□. □□□□□□□")
plot!(plt7[5], df_ode.S, df_ode.I, label=false, color=4,
   linewidth=2, title="□□□□□□ □□□□□□", xlabel=L"S", ylabel=L"I")
plot!(plt7[6], df_ode.t, df_ode.Re, label=L"R_e", color=:green,
    linewidth=2, title=L"R_e(t)", hline=[1.0],
    linestyle=:dash, linecolor=:red)
```



5.12 Сохранение графиков

```
savefig(plt1, plotsdir(script_name, "sir_main.png"))
savefig(plt2, plotsdir(script_name, "sir_infected.png"))
savefig(plt3, plotsdir(script_name, "sir_log_scale.png"))
savefig(plt4, plotsdir(script_name, "sir_percentages.png"))
```

```
savefig(plt5, plotsdir(script_name, "sir_phase_portrait.png"))
savefig(plt6, plotsdir(script_name, "sir_effective_R.png"))
savefig(plt7, plotsdir(script_name, "sir_panel.png"))
```

```
"/home/dabakharev/work/study/2026-1/2026-1==study-simulation-modeli
```

5.13 Бенчмарк производительности

```
println("\n□□□□□□□□ □□□□□□□:")
@benchmark solve(prob_ode, dt=□t)
```

```
□□□□□□□□ □□□□□□□:
```

```
BenchmarkTools.Trial: 10000 samples with 4 evaluations per sample.
 Range (min … max):   7.359 □s …   6.056 ms  □ GC (min … max):  0.00% … 99.
 Time  (median):     8.859 □s               □ GC (median):     0.00%
 Time  (mean ± □):   14.847 □s ± 136.963 □s  □ GC (mean ± □):  25.33% ±  2.7

 □▁□□□□▁▁□□□□□                                                  □
 ▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇□□□□□□□□□□□□□□□□□□□□□□□□□□□▁□□□▁□□□□▁▁▁□▁□□▁□▁□□▁▁▁ ▇
 7.36 □s      Histogram: log(frequency) by time      45.4 □s <

 Memory estimate: 21.85 KiB, allocs estimate: 368.
```

5.14 Дополнительный анализ

```
println("\n== □□□□□□ □□□□□□□□□□□ ==")
println("□□□□□ □□□□□□□□□□□ □□□□□□□□□ (□□□□□□□□): N = ", round(df_ode.N[1], di
println("□□□□□□□ □□□□□ □□□□□□□□□□: I_max = ", round(peak_value, digits=1))
println("□□□□□ □□□□□□□□□□ □□□□: t_peak = ", round(peak_time, digits=1), " □
println("□□□□□□□□ □□□□□ □□□□□□□□□□□□: R(∞) = ", round(df_ode.R[end], digits
println("□□□□ □□□□□□□□□□□□: ", round(df_ode.R[end]/df_ode.N[1]*100, digi

if R0 > 1
    println("\n□□□□□□□□□□□□□ □□□□□□:")
   println(" - □□□□□ □□□□□□□□□□□□□ □□□□□□□□□□: ", round((1 - 1/R0)*100, digit
   println(" - □□□□□□□□□□□□□ □□□ □□□ S/N = 1/R0 = ", round(1/R0, digits=3))
end
```

```
== □□□□□□ □□□□□□□□□□□ ==
□□□□□ □□□□□□□□□□□ □□□□□□□□□ (□□□□□□□□): N = 1000.0
□□□□□□□ □□□□□ □□□□□□□□□□: I_max = 154.8
□□□□□ □□□□□□□□□□ □□□□: t_peak = 19.1 □□□□
□□□□□□□□ □□□□□ □□□□□□□□□□□□: R(∞) = 775.7
□□□□ □□□□□□□□□□□□: 77.6%

□□□□□□□□□□□□□ □□□□□□:
 - □□□□□ □□□□□□□□□□□□□ □□□□□□□□□□: 50.0%
 - □□□□□□□□□□□□□ □□□ □□□ S/N = 1/R0 = 0.5
```

5.15 Сохранение результатов

```
@save datadir(script_name, "sir_results.jld2") df_ode p R0
```

6 Модель Лотки-Вольтерры (хищник-жертва)

Система уравнений:

dx/dt = □ x - □ x y (□□□□□□)
dy/dt = □ x y - □ y (□□□□□□□)

где: - x — популяция жертв - y — популяция хищников - α — естественный прирост жертв - β — коэффициент поедания жертв хищниками - δ — коэффициент конверсии пищи в хищников - γ — естественная смертность хищников

6.1 Активация проекта и загрузка пакетов

```
import Pkg
Pkg.activate("/home/dabakharev/work/study/2026-1/2026-1==study-simu
using DrWatson



using DifferentialEquations
using DataFrames
using StatsPlots
using LaTeXStrings
```

```
using Plots
ENV["GKSwstype"] = "100"
gr(format=:png)
using Statistics
using FFTW
using JLD2

script_name = splitext(basename(PROGRAM_FILE))[1]
mkpath(plotsdir(script_name))
mkpath(datadir(script_name))
```

```
  Activating project at `~/work/study/2026-1/2026-1==study-simulation
```

```
"/home/dabakharev/work/study/2026-1/2026-1==study-simulation-modeli
```

6.2 Определение модели

```
function lotka_volterra!(du, u, p, t)
    x, y = u
    □, □, □, □ = p
    @inbounds begin
        du[1] = □*x - □*x*y
        du[2] = □*x*y - □*y
    end
    nothing
end
```

```
lotka_volterra! (generic function with 1 method)
```

6.3 Параметры и начальные условия

```
p_lv = [0.1,    # □ — □□□□□□□□ □□□□□□□□□□□ □□□□□
        0.02,   # □ — □□□□□□□□ □□□□□□□□ □□□□□
        0.01,   # □ — □□□□□□□□□□□ □□□□□□□□□
        0.3]    # □ — □□□□□□□□□□ □□□□□□□□

u0_lv = [40.0, 9.0]          # [□□□□□□, □□□□□□□]
tspan_lv = (0.0, 200.0)      # □□□□□□□□□□□□ □□□□□□□□□
dt_lv = 0.01                 # □□□ □□□□□□□□□□□□□□□

prob_lv = ODEProblem(lotka_volterra!, u0_lv, tspan_lv, p_lv)
sol_lv = solve(prob_lv, Tsit5(); reltol=1e-8, abstol=1e-10, saveat=0.
```

```
retcode: Success
Interpolation: 1st order linear
t: 2001-element Vector{Float64}:
   0.0
   0.1
   0.2
   0.3
   0.4
   0.5
   0.6
   0.7
   0.8
   0.9
   1.0
   1.1
```

```
   1.2
  ⋮
 198.9
 199.0
 199.1
 199.2
 199.3
 199.4
 199.5
 199.6
 199.7
 199.8
 199.9
 200.0
u: 2001-element Vector{Vector{Float64}}:
 [40.0, 9.0]
 [39.677732019054936, 9.088990237143179]
 [39.35113789515877, 9.175882726251176]
 [39.02053587742845, 9.260562174604983]
 [38.68624820359512, 9.34291631164816]
 [38.34860018549613, 9.422836266192155]
 [38.007919321489645, 9.500216918207974]
 [37.66453436579186, 9.574957251506675]
 [37.31877446910066, 9.646960666011953]
 [36.97096821728331, 9.716135308386768]
 [36.6214428710377, 9.782394322077632]
 [36.270523370595285, 9.845656152960181]
 [35.91853162632679, 9.905844752781562]
 ⋮
```

```
 [17.88967563585216, 5.712401889017617]
 [17.865436241348924, 5.643570487712594]
 [17.84367345156099, 5.575440227126908]
 [17.82435314446551, 5.508019317739071]
 [17.80744203801905, 5.441315149286218]
 [17.79290768891314, 5.3753342945187415]
 [17.78071847297987, 5.310082579284753]
 [17.77084358542412, 5.245565092116128]
 [17.763253011515754, 5.181786247592668]
 [17.75791751894706, 5.11874980604259]
 [17.75480865783273, 5.056458873542604]
 [17.753898747815335, 4.994915937021203]
```

6.4 Подготовка данных

```
df_lv = DataFrame()
df_lv[!, :t] = sol_lv.t
df_lv[!, :prey] = [u[1] for u in sol_lv.u]
df_lv[!, :predator] = [u[2] for u in sol_lv.u]
```

```
2001-element Vector{Float64}:
 9.0
 9.088990237143179
 9.175882726251176
 9.260562174604983
 9.34291631164816
 9.422836266192155
 9.500216918207974
```

```
 9.574957251506675
 9.646960666011953
 9.716135308386768
 9.782394322077632
 9.845656152960181
 9.905844752781562
 
 5.712401889017617
 5.643570487712594
 5.575440227126908
 5.508019317739071
 5.441315149286218
 5.3753342945187415
 5.310082579284753
 5.245565092116128
 5.181786247592668
 5.11874980604259
 5.056458873542604
 4.994915937021203
```

Производные

```
df_lv[!, :dprey_dt] = p_lv[1] .* df_lv.prey .- p_lv[2] .* df_lv.prey .*
df_lv[!, :dpredator_dt] = p_lv[3] .* df_lv.prey .* df_lv.predator .- p
```

```
2001-element Vector{Float64}:
  0.9000000000000004
  0.8796081183887825
  0.8580494468298019
  0.8353523334118109
```

```
  0.8115489002839338
  0.7866749259983155
  0.7607697061766046
  0.7338758890516086
  0.7060392942682663
  0.677308704295883
  0.6477356514560064
  0.6173741700598212
  0.5862805545482095
  □
 -0.6917903977437418
 -0.6848226590959001
 -0.6777687205225765
 -0.6706369808625294
 -0.6634355034707723
 -0.6561720193614122
 -0.6488539396800559
 -0.6414883599432727
 -0.6340820726019867
 -0.6266415732544711
 -0.6191730642032875
 -0.6116824631091249
```

Относительные изменения

```
df_lv[!, :prey_pct_change] = df_lv.dprey_dt ./ df_lv.prey .* 100
df_lv[!, :predator_pct_change] = df_lv.dpredator_dt ./ df_lv.predator
```

```
2001-element Vector{Float64}:
  10.000000000000004
```

9.677732019054936
9.351137895158775
9.02053587742845
8.68624820359512
8.348600185496137
8.007919321489647
7.664534365791859
7.318774469100665
6.970968217283306
6.6214428710377025
6.270523370595289
5.918531626326789
-12.11032436414784
-12.134563758651073
-12.15632654843901
-12.17564685553449
-12.192557961980947
-12.207092311086857
-12.21928152702013
-12.22915641457588
-12.236746988484247
-12.242082481052936
-12.245191342167269
-12.246101252184664

6.5 Вывод информации о модели

```
println("="^60)
println("□□□□□□ □□□□□-□□□□□□□□□ (□□□□□□-□□□□□□)")
println("="^60)
println("\n□□□□□□□□ □□□□□□:")
println("□ (□□□□□□□ □□□□□□□□□□ □□□□□) = ", p_lv[1])
println("□ (□□□□□□□ □□□□□□□ □□□□□) = ", p_lv[2])
println("□ (□□□□□□□□□ □□□□□□□□) = ", p_lv[3])
println("□ (□□□□□□□□□ □□□□□□□□) = ", p_lv[4])
println("\n□□□□□□□□ □□□□□□□:")
println("□□□□□□ (x₀) = ", u0_lv[1])
println("□□□□□□□ (y₀) = ", u0_lv[2])
```

```
============================================================
□□□□□□ □□□□□-□□□□□□□□□ (□□□□□□-□□□□□□)
============================================================

□□□□□□□□ □□□□□□:
□ (□□□□□□□ □□□□□□□□□□ □□□□□) = 0.1
□ (□□□□□□□ □□□□□□□ □□□□□) = 0.02
□ (□□□□□□□□□ □□□□□□□□) = 0.01
□ (□□□□□□□□□ □□□□□□□□) = 0.3

□□□□□□□□ □□□□□□□:
□□□□□□ (x₀) = 40.0
□□□□□□□ (y₀) = 9.0
```

6.6 Стационарные точки

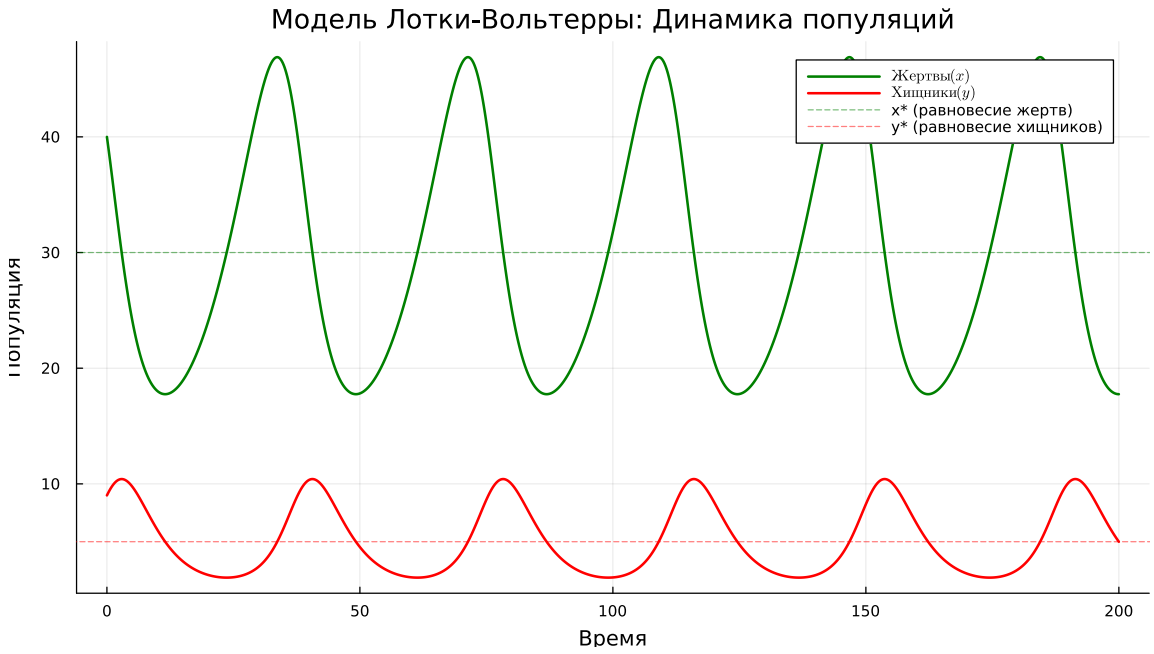
```
x_star = p_lv[4] / p_lv[3]   # x* = □/□
y_star = p_lv[1] / p_lv[2]   # y* = □/□
println("\n□□□□□□□□□□□□ □□□□□ (□□□□□□□□□ □□□□□□□□□□):")
println("x* = □/□ = ", round(x_star, digits=3))
println("y* = □/□ = ", round(y_star, digits=3))
```

```
□□□□□□□□□□□□ □□□□□ (□□□□□□□□□ □□□□□□□□□□):
x* = □/□ = 30.0
y* = □/□ = 5.0
```

6.7 График 1: динамика популяций

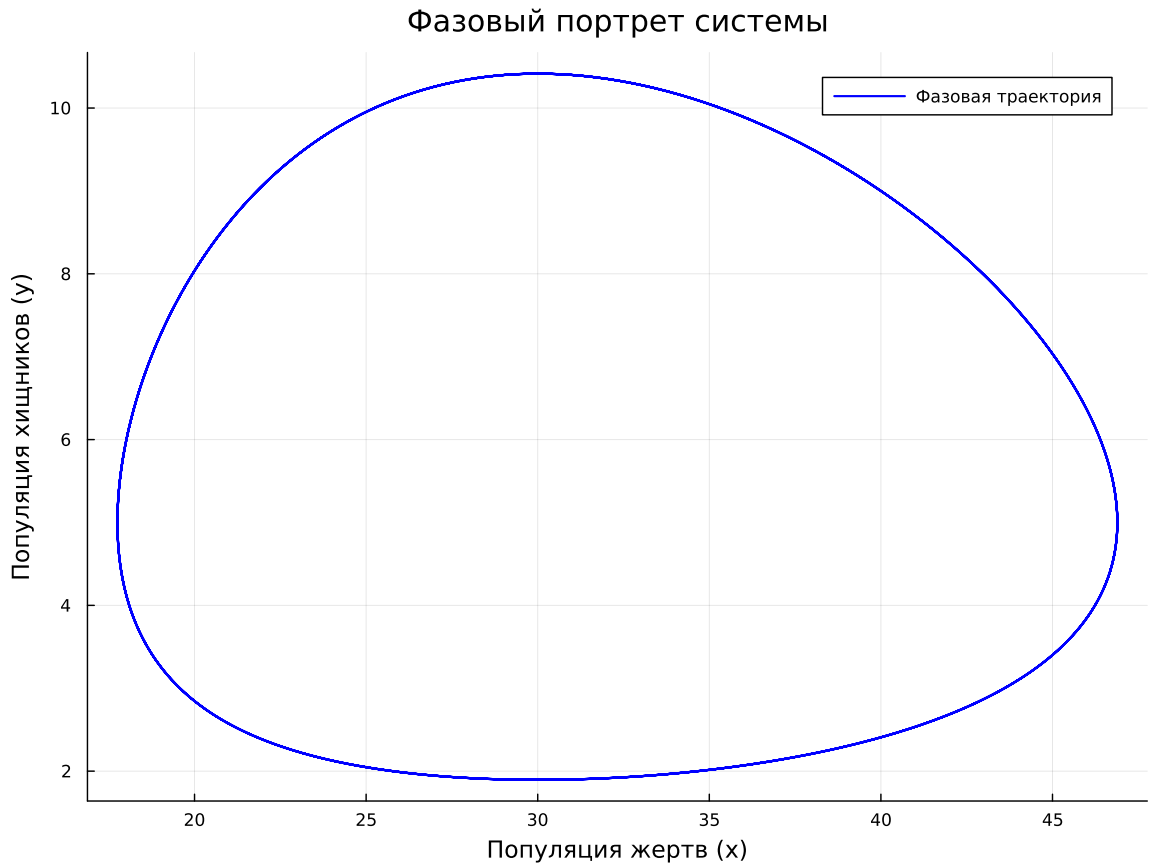
```
plt1 = plot(df_lv.t, [df_lv.prey df_lv.predator],
    label=[L"□□□□□□ (x)" L"□□□□□□□ (y)"],
    xlabel="□□□□□", ylabel="□□□□□□□□□",
    title="□□□□□□ □□□□□-□□□□□□□□□: □□□□□□□□ □□□□□□□□□",
   linewidth=2, legend=:topright, grid=true, size=(900, 500),
    color=[:green :red])

hline!(plt1, [x_star], color=:green, linestyle=:dash, alpha=0.5,
    label="x* (□□□□□□□□□□ □□□□□)")
hline!(plt1, [y_star], color=:red, linestyle=:dash, alpha=0.5,
    label="y* (□□□□□□□□□□ □□□□□□□□)")
```



6.8 График 2: фазовый портрет с изоклинами и стрелками

```
plt2 = plot(df_lv.prey, df_lv.predator,
    label="□□□□□□□ □□□□□□□□□□",
   xlabel="□□□□□□□□□ □□□□□ (x)", ylabel="□□□□□□□□□ □□□□□□□□ (y)",
    title="□□□□□□□ □□□□□□□ □□□□□□□",
   color=:blue, linewidth=1.5, grid=true, size=(800, 600),
    legend=:topright)
```

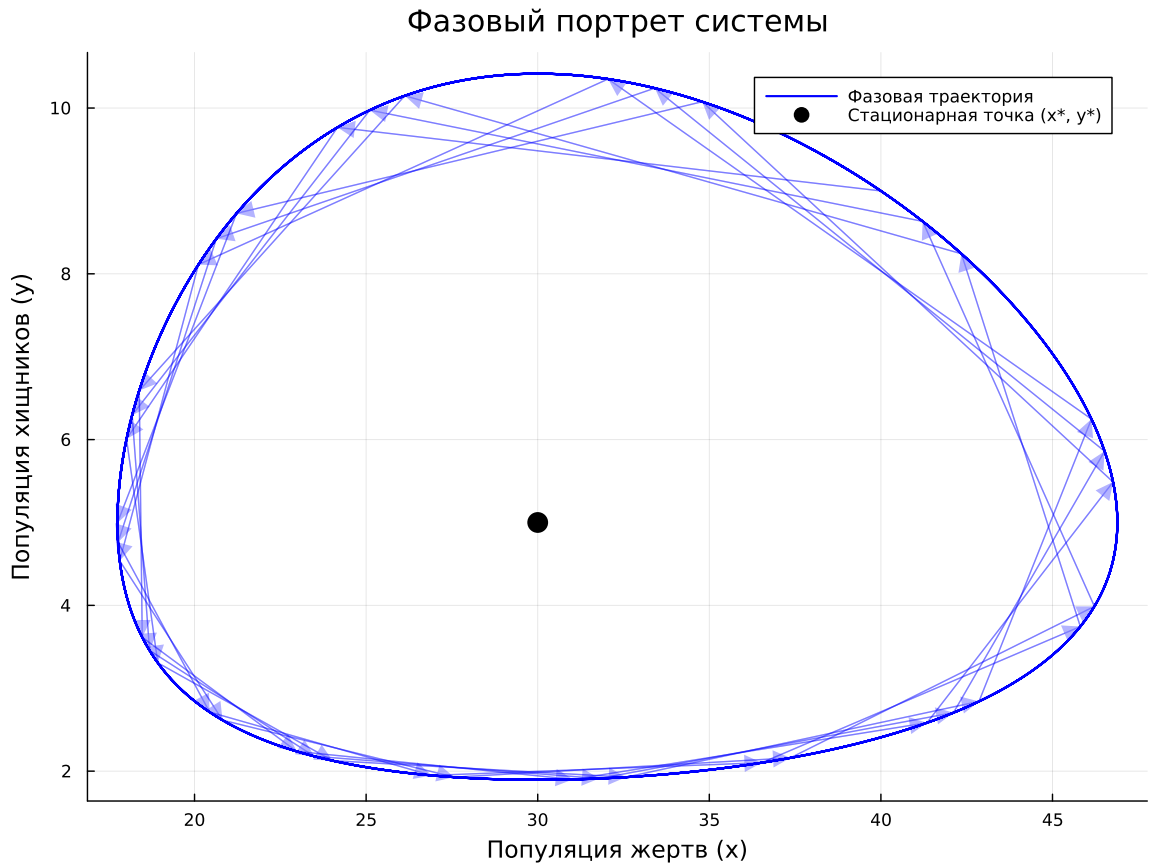


Стрелки направления

```
step = 50
for i in 1:step:length(df_lv.prey)-step
    plot!(plt2, [df_lv.prey[i], df_lv.prey[i+step]],
              [df_lv.predator[i], df_lv.predator[i+step]],
      arrow=:closed, color=:blue, alpha=0.3, label=false)
end
```

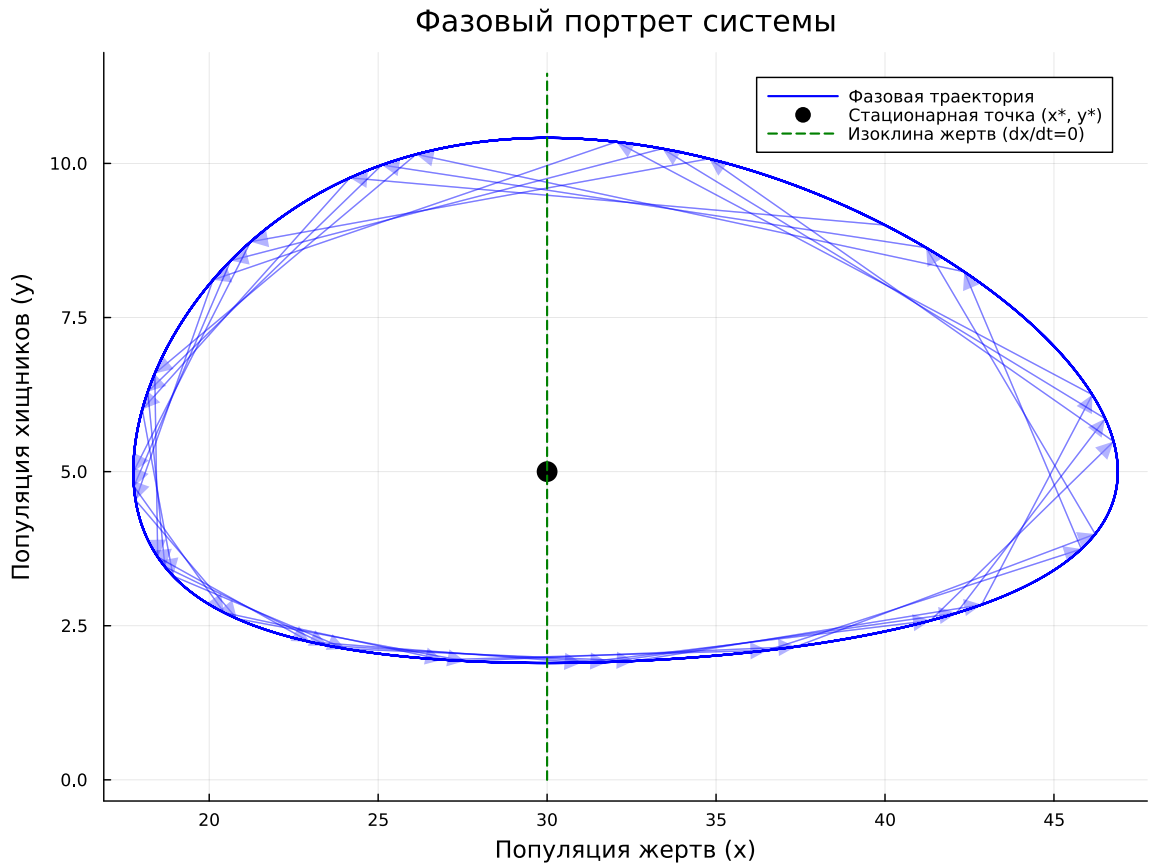
Стационарная точка

```
scatter!(plt2, [x_star], [y_star], color=:black, markersize=8,
    label="□□□□□□□□□□□ □□□□ (x*, y*)")
```



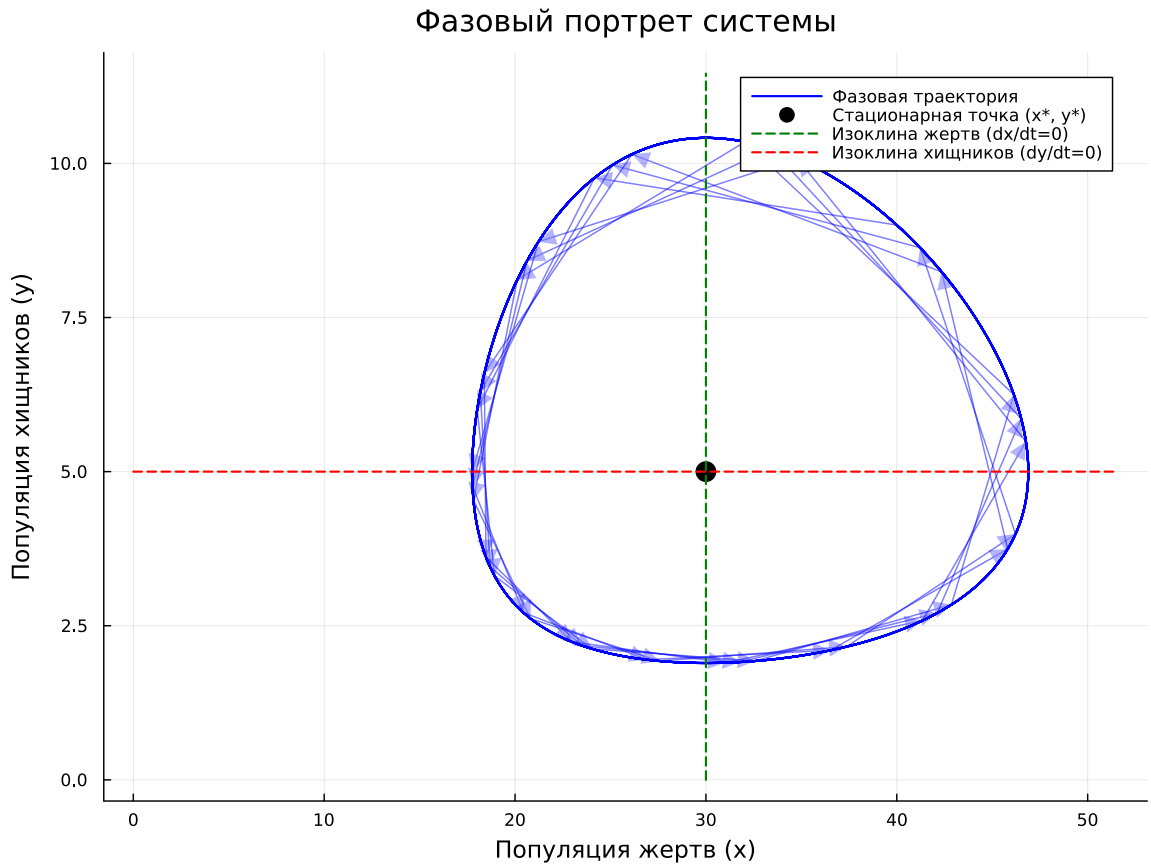
Изоклина хищников (dy/dt = 0): x = γ/δ = x_star (вертикальная линия)

```
y_range = LinRange(0, maximum(df_lv.predator)*1.1, 100)
x_nulcline = fill(x_star, length(y_range))
plot!(plt2, x_nulcline, y_range, color=:green, linestyle=:dash,
    linewidth=1.5, label="□□□□□□□□ □□□□□ (dx/dt=0)")
```



Изоклина жертв (dx/dt = 0): y = α/β = y_star (горизонтальная линия)

```
x_range = LinRange(0, maximum(df_lv.prey)*1.1, 100)
y_nulcline = fill(y_star, length(x_range))
plot!(plt2, x_range, y_nulcline, color=:red, linestyle=:dash,
    linewidth=1.5, label="□□□□□□□□ □□□□□□□□ (dy/dt=0)")
```



6.9 График 3: производные популяций

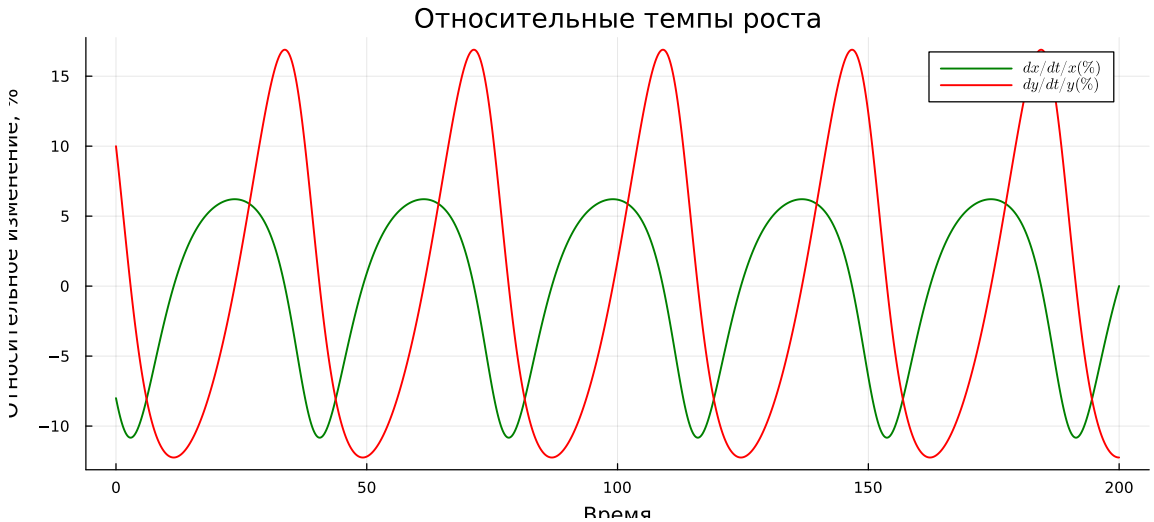
```
plt3 = plot(df_lv.t, [df_lv.dprey_dt df_lv.dpredator_dt],
    label=[L"dx/dt" L"dy/dt"],
    xlabel="□□□□□", ylabel="□□□□□□□□ □□□□□□□□□",
    title="□□□□□□□□□□□ □□□□□□□□□",
   linewidth=1.5, legend=:topright, grid=true, size=(900, 400),
    color=[:green :red])

hline!(plt3, [0], color=:black, linestyle=:solid, alpha=0.3, label=fa
```



6.10 График 4: относительные изменения (%)

```
plt4 = plot(df_lv.t, [df_lv.prey_pct_change df_lv.predator_pct_change
    label=[L"dx/dt / x (\%)" L"dy/dt / y (\%)"],
    xlabel="□□□□□", ylabel="□□□□□□□□□□□□ □□□□□□□□□, %",
    title="□□□□□□□□□□□□ □□□□□ □□□□□",
   linewidth=1.5, legend=:topright, grid=true, size=(900, 400),
    color=[:green :red])
```

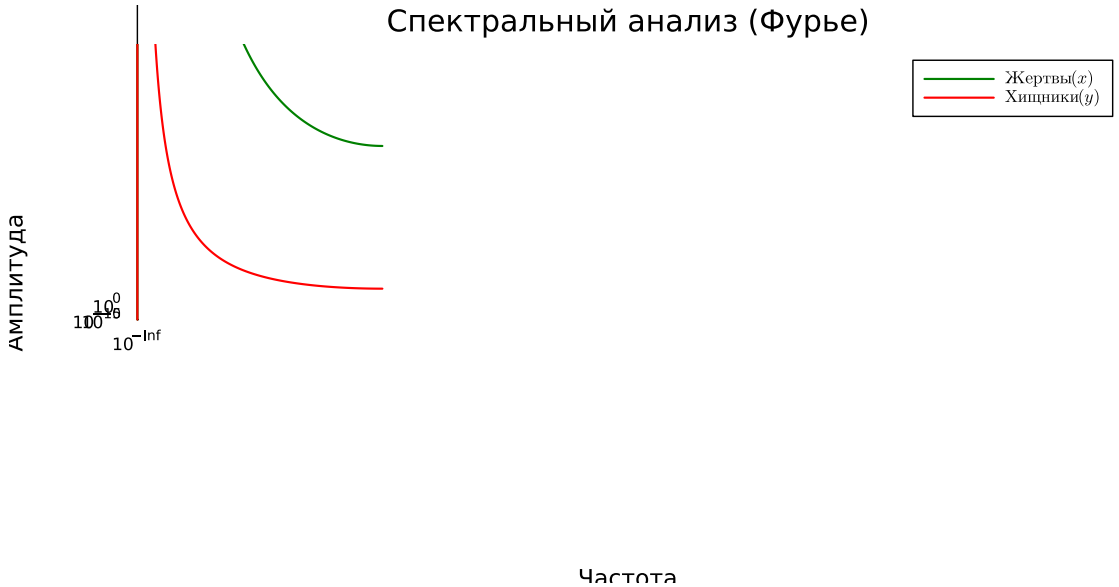


6.11 График 5: спектральный анализ (Фурье)

```
function compute_fft(signal, dt)
    n = length(signal)
    spectrum = abs.(rfft(signal))
    freq = rfftfreq(n, 1/dt)
    return freq, spectrum
end

freq_prey, spectrum_prey = compute_fft(df_lv.prey .- mean(df_lv.prey)
freq_predator, spectrum_predator = compute_fft(df_lv.predator .- mean

plt5 = plot(freq_prey, [spectrum_prey spectrum_predator],
    label=[L"□□□□□□ (x)" L"□□□□□□□ (y)"],
    xlabel="□□□□□□□", ylabel="□□□□□□□□□",
    title="□□□□□□□□□□□ □□□□□□ (□□□□□)",
    linewidth=1.5, xscale=:log10, yscale=:log10,
    legend=:topright, grid=true, size=(800, 400),
    color=[:green :red])
```




```
┌ Warning: No strict ticks found
└ @ PlotUtils ~/.julia/packages/PlotUtils/k30nd/src/ticks.jl:194
┌ Warning: No strict ticks found
└ @ PlotUtils ~/.julia/packages/PlotUtils/k30nd/src/ticks.jl:194
┌ Warning: Invalid negative or zero value 0.0 found at series index 1 fo
└ @ Plots ~/.julia/packages/Plots/h49MV/src/utils.jl:105
┌ Warning: Invalid negative or zero value 0.0 found at series index 1 fo
└ @ Plots ~/.julia/packages/Plots/h49MV/src/utils.jl:105
┌ Warning: No strict ticks found
└ @ PlotUtils ~/.julia/packages/PlotUtils/k30nd/src/ticks.jl:194
┌ Warning: No strict ticks found
└ @ PlotUtils ~/.julia/packages/PlotUtils/k30nd/src/ticks.jl:194
┌ Warning: Invalid negative or zero value 0.0 found at series index 1 fo
└ @ Plots ~/.julia/packages/Plots/h49MV/src/utils.jl:105
┌ Warning: Invalid negative or zero value 0.0 found at series index 1 fo
└ @ Plots ~/.julia/packages/Plots/h49MV/src/utils.jl:105
┌ Warning: No strict ticks found
└ @ PlotUtils ~/.julia/packages/PlotUtils/k30nd/src/ticks.jl:194
```

```
┌ Warning: No strict ticks found
└ @ PlotUtils ~/.julia/packages/PlotUtils/k30nd/src/ticks.jl:194
┌ Warning: No strict ticks found
└ @ PlotUtils ~/.julia/packages/PlotUtils/k30nd/src/ticks.jl:194
┌ Warning: Invalid negative or zero value 0.0 found at series index 1 fo
└ @ Plots ~/.julia/packages/Plots/h49MV/src/utils.jl:105
┌ Warning: Invalid negative or zero value 0.0 found at series index 1 fo
└ @ Plots ~/.julia/packages/Plots/h49MV/src/utils.jl:105
┌ Warning: No strict ticks found
└ @ PlotUtils ~/.julia/packages/PlotUtils/k30nd/src/ticks.jl:194
┌ Warning: No strict ticks found
└ @ PlotUtils ~/.julia/packages/PlotUtils/k30nd/src/ticks.jl:194
┌ Warning: Invalid negative or zero value 0.0 found at series index 1 fo
└ @ Plots ~/.julia/packages/Plots/h49MV/src/utils.jl:105
┌ Warning: Invalid negative or zero value 0.0 found at series index 1 fo
└ @ Plots ~/.julia/packages/Plots/h49MV/src/utils.jl:105
```

Доминирующая частота

```
if length(spectrum_prey) > 0
    idx_prey = argmax(spectrum_prey[2:end]) + 1
    dominant_freq_prey = freq_prey[idx_prey]
    period_prey = 1 / dominant_freq_prey
    println("\n□□□□□□□□□□□□ □□□□□□□ □□□□□□□□□ □□□□□: ",
        round(dominant_freq_prey, digits=4), " □□")
  println("□□□□□□ □□□□□□□□□ □□□□□: ", round(period_prey, digits=2),
        " □□□□□□ □□□□□□□")
end
```

□□□□□□□□□□□□ □□□□□□□ □□□□□□□□□ □□□□□: 0.2499 □□

□□□□□□ □□□□□□□□□ □□□□□: 4.0 □□□□□□ □□□□□□□

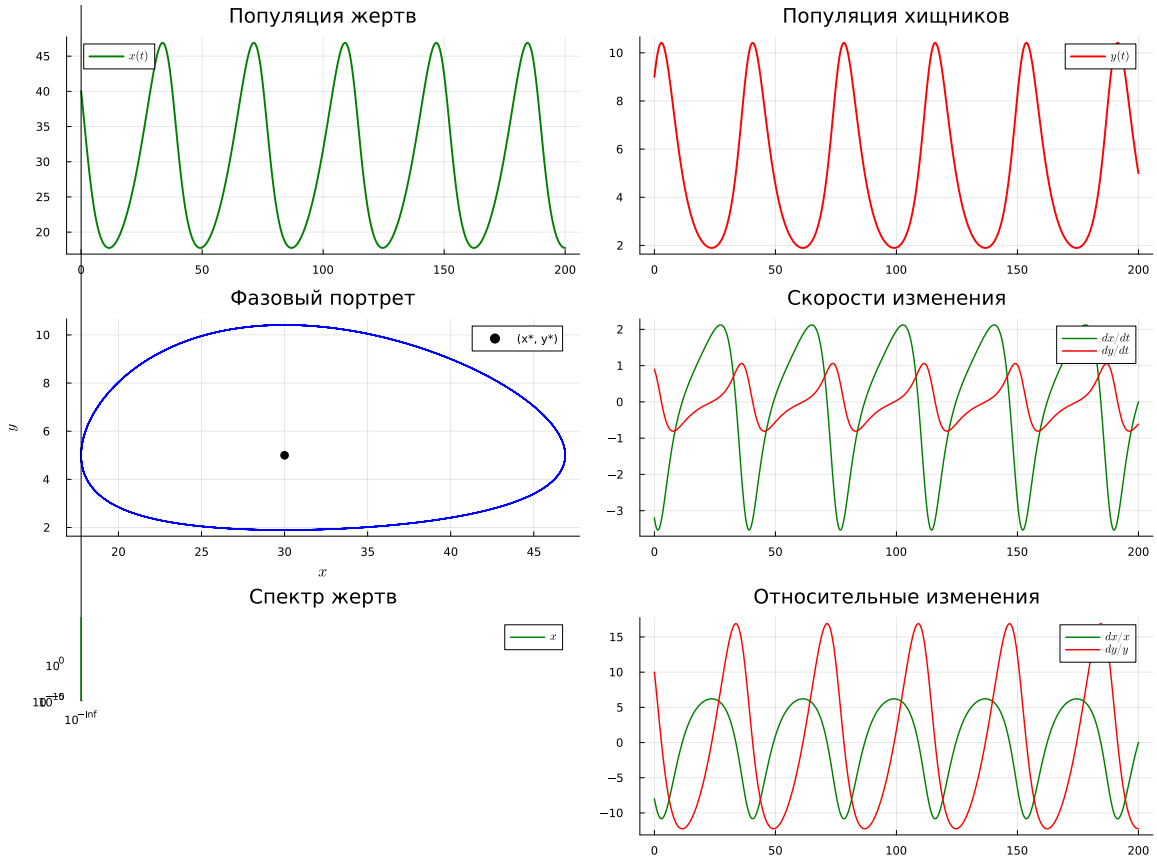
6.12 График 6: компактная панель всех графиков

```
plt6 = plot(layout=(3, 2), size=(1200, 900))

plot!(plt6[1], df_lv.t, df_lv.prey, label=L"x(t)", color=:green,
    linewidth=2, title="□□□□□□□□□ □□□□□", grid=true)
plot!(plt6[2], df_lv.t, df_lv.predator, label=L"y(t)", color=:red,
    linewidth=2, title="□□□□□□□□□ □□□□□□□□", grid=true)
plot!(plt6[3], df_lv.prey, df_lv.predator, label=false, color=:blue,
   linewidth=1.5, title="□□□□□□□ □□□□□□□", xlabel=L"x", ylabel=L"y", gri
scatter!(plt6[3], [x_star], [y_star], color=:black, markersize=5,
    label="(x*, y*)")
plot!(plt6[4], df_lv.t, [df_lv.dprey_dt df_lv.dpredator_dt],
   label=[L"dx/dt" L"dy/dt"], color=[:green :red], linewidth=1.5,
    title="□□□□□□□□ □□□□□□□□□", grid=true, legend=:topright)
plot!(plt6[5], freq_prey, spectrum_prey, label=L"x", color=:green,
   linewidth=1.5, title="□□□□□□ □□□□□", xscale=:log10, yscale=:log10, gr
plot!(plt6[6], df_lv.t, [df_lv.prey_pct_change df_lv.predator_pct_ch
   label=[L"dx/x" L"dy/y"], color=[:green :red], linewidth=1.5,
   title="□□□□□□□□□□□□□ □□□□□□□□□", grid=true, legend=:topright)
```

```
┌ Warning: No strict ticks found
└ @ PlotUtils ~/.julia/packages/PlotUtils/k30nd/src/ticks.jl:194
┌ Warning: No strict ticks found
└ @ PlotUtils ~/.julia/packages/PlotUtils/k30nd/src/ticks.jl:194
```

```
┌ Warning: Invalid negative or zero value 0.0 found at series index 1 fo
└ @ Plots ~/.julia/packages/Plots/h49MV/src/utils.jl:105
┌ Warning: No strict ticks found
└ @ PlotUtils ~/.julia/packages/PlotUtils/k30nd/src/ticks.jl:194
┌ Warning: No strict ticks found
└ @ PlotUtils ~/.julia/packages/PlotUtils/k30nd/src/ticks.jl:194
┌ Warning: Invalid negative or zero value 0.0 found at series index 1 fo
└ @ Plots ~/.julia/packages/Plots/h49MV/src/utils.jl:105
┌ Warning: No strict ticks found
└ @ PlotUtils ~/.julia/packages/PlotUtils/k30nd/src/ticks.jl:194
┌ Warning: No strict ticks found
└ @ PlotUtils ~/.julia/packages/PlotUtils/k30nd/src/ticks.jl:194
┌ Warning: No strict ticks found
└ @ PlotUtils ~/.julia/packages/PlotUtils/k30nd/src/ticks.jl:194
┌ Warning: Invalid negative or zero value 0.0 found at series index 1 fo
└ @ Plots ~/.julia/packages/Plots/h49MV/src/utils.jl:105
```




```
┌ Warning: No strict ticks found
└ @ PlotUtils ~/.julia/packages/PlotUtils/k30nd/src/ticks.jl:194
┌ Warning: No strict ticks found
└ @ PlotUtils ~/.julia/packages/PlotUtils/k30nd/src/ticks.jl:194
┌ Warning: Invalid negative or zero value 0.0 found at series index 1 fo
└ @ Plots ~/.julia/packages/Plots/h49MV/src/utils.jl:105
```

6.13 Сохранение графиков

```
savefig(plt1, plotsdir(script_name, "lv_dynamics.png"))
savefig(plt2, plotsdir(script_name, "lv_phase_portrait.png"))
savefig(plt3, plotsdir(script_name, "lv_derivatives.png"))
```

```
savefig(plt4, plotsdir(script_name, "lv_relative_changes.png"))
savefig(plt5, plotsdir(script_name, "lv_spectrum.png"))
savefig(plt6, plotsdir(script_name, "lv_panel.png"))
```

```
┌ Warning: No strict ticks found
└ @ PlotUtils ~/.julia/packages/PlotUtils/k30nd/src/ticks.jl:194
┌ Warning: No strict ticks found
└ @ PlotUtils ~/.julia/packages/PlotUtils/k30nd/src/ticks.jl:194
┌ Warning: Invalid negative or zero value 0.0 found at series index 1 fo
└ @ Plots ~/.julia/packages/Plots/h49MV/src/utils.jl:105
┌ Warning: Invalid negative or zero value 0.0 found at series index 1 fo
└ @ Plots ~/.julia/packages/Plots/h49MV/src/utils.jl:105
┌ Warning: No strict ticks found
└ @ PlotUtils ~/.julia/packages/PlotUtils/k30nd/src/ticks.jl:194
┌ Warning: No strict ticks found
└ @ PlotUtils ~/.julia/packages/PlotUtils/k30nd/src/ticks.jl:194
┌ Warning: Invalid negative or zero value 0.0 found at series index 1 fo
└ @ Plots ~/.julia/packages/Plots/h49MV/src/utils.jl:105

"/home/dabakharev/work/study/2026-1/2026-1==study-simulation-modeli
```

6.14 Анализ результатов

```
println("\n" * "="^60)
println("□□□□□□ □□□□□□□□□□□□")
println("="^60)

println("\n□□□□□□□□ □□□□□□□□□□:")
```

```
println("□□□□□□: min = ", round(minimum(df_lv.prey), digits=2),
        ", max = ", round(maximum(df_lv.prey), digits=2),
        ", mean = ", round(mean(df_lv.prey), digits=2))
println("□□□□□□□: min = ", round(minimum(df_lv.predator), digits=2),
      ", max = ", round(maximum(df_lv.predator), digits=2),
       ", mean = ", round(mean(df_lv.predator), digits=2))
```

```
============================================================
□□□□□□ □□□□□□□□□□
============================================================

□□□□□□□□ □□□□□□□□□□:
□□□□□□: min = 17.75, max = 46.89, mean = 29.71
□□□□□□□: min = 1.9, max = 10.41, mean = 5.2
```

Поиск первого пика

```
function find_first_peak(signal, time)
    for i in 2:length(signal)-1
      if signal[i] > signal[i-1] && signal[i] > signal[i+1]
            return time[i], signal[i]
        end
    end
    return NaN, NaN
end

peak_time_prey, peak_value_prey = find_first_peak(df_lv.prey, df_lv.t
peak_time_predator, peak_value_predator = find_first_peak(df_lv.preda
```

```
if !isnan(peak_time_prey) && !isnan(peak_time_predator)
    phase_shift = peak_time_predator - peak_time_prey
    println("\n[illegible]:")
  println("[illegible] [illegible]: [illegible] = ", round(peak_time_prey, digits=2),
          ", [illegible] = ", round(peak_value_prey, digits=2))
  println("[illegible] [illegible]: [illegible] = ", round(peak_time_predator, digit
        ", [illegible] = ", round(peak_value_predator, digits=2))
  println("[illegible] [illegible] ([illegible] [illegible]): ", round(phase_shift, digits=2))
end
```

```
[illegible]:
[illegible] [illegible]: [illegible] = 33.7, [illegible] = 46.89
[illegible] [illegible]: [illegible] = 2.9, [illegible] = 10.41
[illegible] [illegible] ([illegible] [illegible]): -30.8
```

6.15 Сохранение результатов

```
@save datadir(script_name, "lv_results.jld2") df_lv p_lv u0_lv

println("\n[illegible] [illegible] [illegible]!")
```

```
[illegible] [illegible] [illegible]!
```

7 Параметрическое исследование модели SIR

Исследуем влияние коэффициента заражения β и скорости выздоровления γ на динамику эпидемии: пик, время пика, итоговое число переболевших.

7.1 Активация проекта и загрузка пакетов

```
import Pkg
Pkg.activate("/home/dabakharev/work/study/2026-1/2026-1==study-simu
using DrWatson
using DifferentialEquations
using DataFrames
using Plots
using JLD2
using BenchmarkTools

ENV["GKSwstype"] = "100"
gr(format=:png)

script_name = splitext(basename(PROGRAM_FILE))[1]
mkpath(plotsdir(script_name))
```

```
mkpath(datadir(script_name))
```

```
  Activating project at `~/work/study/2026-1/2026-1==study-simulation
```

```
"/home/dabakharev/work/study/2026-1/2026-1==study-simulation-modeli
```

7.2 Определение модели SIR

$$
\begin{aligned}
dS/dt &= -\beta c I S / N \\
dI/dt &= \beta c I S / N - \gamma I \\
dR/dt &= \gamma I
\end{aligned}
$$

```
function sir_ode!(du, u, p, t)
    (S, I, R) = u
    (β, c, γ) = p
    N = S + I + R
    @inbounds begin
        du[1] = -β * c * I / N * S
        du[2] = β * c * I / N * S - γ * I
        du[3] = γ * I
    end
    nothing
end
```

```
sir_ode! (generic function with 1 method)
```

7.3 Базовые параметры эксперимента

Все параметры собраны в `Dict` для систематизации.

```
base_params = Dict(
    :u0    => [[990.0, 10.0, 0.0]],  # S0, I0, R0
   :□    => 0.05,                 # □□□□□□□□□□□ □□□□□□□□ □□□ □□□□□□□□
    :c     => 10.0,                       # □□□□□□□ □□□□□ □□□□□□□□□
    :□     => 0.25,                        # □□□□□□□□ □□□□□□□□□□□□□
    :tspan => [(0.0, 40.0)],               # □□□□□□□□□ □□□□□□□□
    :solver => [Tsit5()],                  # □□□□□ □□□□□□□
    :saveat => [0.1]                       # □□□ □□□□□□□□□□
)

println("□□□□□□□ □□□□□□□□□ □□□□□□□□□□□□:")
for (key, value) in base_params
    println("  $key = $value")
end
```

□□□□□□□ □□□□□□□□□ □□□□□□□□□□□□:

```
  □ = 0.25
  u0 = [[990.0, 10.0, 0.0]]
  saveat = [0.1]
  solver = [Tsit5{typeof(OrdinaryDiffEqCore.trivial_limiter!), typeof
  c = 10.0
  □ = 0.05
  tspan = [(0.0, 40.0)]
```

7.4 Функция запуска одного эксперимента

```
function run_sir(params)
   prob = ODEProblem(sir_ode!, params[:u0], params[:tspan],
         (params[:β], params[:c], params[:γ]))
   sol = solve(prob, params[:solver]; saveat=params[:saveat])

    I = [u[2] for u in sol.u]
    R = [u[3] for u in sol.u]
    R0 = params[:c] * params[:β] / params[:γ]

    return Dict(
        "t"       => sol.t,
        "S"       => [u[1] for u in sol.u],
        "I"       => I,
        "R"       => R,
        "I_max"   => maximum(I),
        "t_peak"  => sol.t[argmax(I)],
        "R_inf"   => R[end],
        "R0"      => R0
    )
end
```

```
run_sir (generic function with 1 method)
```

7.5 Запуск базового эксперимента (одна комбинация)

```
base_params_single = dict_list(base_params)[1]
data, path = produce_or_load(
    datadir(script_name, "single"),
    base_params_single,
    run_sir;
    prefix = "sir_base",
    tag = false,
    verbose = true
)

println("\n□□□□□□□□□□ □□□□□□□□ □□□□□□□□□□□□:")
println("  R0            = ", round(data["R0"], digits=3))
println("  I_max         = ", round(data["I_max"], digits=1))
println("  t_peak        = ", round(data["t_peak"], digits=1), " □□□□")
println("  R(∞)          = ", round(data["R_inf"], digits=1))
println("  □□□□ □□□□□□□□□□□: ", path)
```

```
□□□□□□□□□□ □□□□□□□□ □□□□□□□□□□□□:
  R0              = 2.0
  I_max           = 158.5
  t_peak          = 17.5 □□□□
  R(∞)            = 775.7
  □□□□ □□□□□□□□□□□: /home/dabakharev/work/study/2026-1/2026-1==study-sim
```

7.6 Визуализация базового эксперимента

```
p1 = plot(data["t"], data["I"],
   label="I(t), □=$(base_params[:□]), □=$(base_params[:□])",
    xlabel="□□□□□, □□□", ylabel="□□□□□□□□□□□□□□□",
    title="SIR: □□□□□□□ □□□□□□□□□□□",
    lw=2, legend=:topleft, grid=true)
savefig(plotsdir(script_name, "sir_single.png"))
```

```
"/home/dabakharev/work/study/2026-1/2026-1==study-simulation-modeli
```

7.7 Параметрическое сканирование по β и γ

```
param_grid = Dict(
    :u0    => [[990.0, 10.0, 0.0]],
    :□     => [0.03, 0.05, 0.08],      # □□□□□□□□□□□ □
    :c     => [10.0],                  # □□□□□□□□□ c
    :□     => [0.1, 0.25, 0.4],        # □□□□□□□□□□□ □
    :tspan => [(0.0, 40.0)],
    :solver => [Tsit5()],
    :saveat => [0.1]
)

all_params = dict_list(param_grid)

println("\n" * "="^60)
println("□□□□□□□□□□□□□□□ □□□□□□□□□□□ SIR")
println("□□□□□ □□□□□□□□□□: ", length(all_params))
```

```
println("□□□□□□□□□□□□ □: ", param_grid[:□])
println("□□□□□□□□□□□□ □: ", param_grid[:□])
println("="^60)
```

```
============================================================
□□□□□□□□□□□□□□□ □□□□□□□□□□□ SIR
□□□□□ □□□□□□□□□□: 9
□□□□□□□□□□□ □: [0.03, 0.05, 0.08]
□□□□□□□□□□□ □: [0.1, 0.25, 0.4]
============================================================
```

7.8 Запуск всех экспериментов и сбор сводных результатов

```
all_results = []
all_dfs = []

for (i, params) in enumerate(all_params)
   println("□□□□□□□□: $i/$(length(all_params)) | □=$(params[:□]), □=$(pa
    data, path = produce_or_load(
        datadir(script_name, "scan"),
        params,
        run_sir;
        prefix = "sir_scan",
        tag = false,
        verbose = false
    )
```

```
    result_summary = merge(params, Dict(
        :R0      => data["R0"],
        :I_max   => data["I_max"],
        :t_peak  => data["t_peak"],
        :R_inf   => data["R_inf"],
        :filepath => path
    ))
    push!(all_results, result_summary)

    df = DataFrame(
        t = data["t"],
        S = data["S"],
        I = data["I"],
        R = data["R"],
        □ = fill(params[:□], length(data["t"])),
        □ = fill(params[:□], length(data["t"]))
    )
    push!(all_dfs, df)
end
```

□□□□□□□□: 1/9 | □=0.03, □=0.1
□□□□□□□□: 2/9 | □=0.03, □=0.25
□□□□□□□□: 3/9 | □=0.03, □=0.4
□□□□□□□□: 4/9 | □=0.05, □=0.1
□□□□□□□□: 5/9 | □=0.05, □=0.25
□□□□□□□□: 6/9 | □=0.05, □=0.4
□□□□□□□□: 7/9 | □=0.08, □=0.1
□□□□□□□□: 8/9 | □=0.08, □=0.25
□□□□□□□□: 9/9 | □=0.08, □=0.4

7.9 Сводная таблица

```
results_df = DataFrame(all_results)
println("\n□□□□□□□ □□□□□□□ □□□□□□□□□□:")
println(results_df[:, [:□, :□, :R0, :I_max, :t_peak, :R_inf]])
```

```
□□□□□□□ □□□□□□□ □□□□□□□□□□:
9×6 DataFrame
 Row □ □        □         R0        I_max     t_peak    R_inf
     □ Float64  Float64   Float64   Float64   Float64   Float64
─────┼──────────────────────────────────────────────────────────
   1 □    0.03     0.1       3.0   303.813      26.6   700.529
   2 □    0.03     0.25      1.2    23.1073     32.2   187.764
   3 □    0.03     0.4       0.75   10.0         0.0    36.8878
   4 □    0.05     0.1       5.0   480.127      15.3   921.529
   5 □    0.05     0.25      2.0   158.452      17.5   775.694
   6 □    0.05     0.4       1.25   29.5254     20.0   345.755
   7 □    0.08     0.1       8.0   616.352       9.6   962.171
   8 □    0.08     0.25      3.2   327.154       9.9   951.808
   9 □    0.08     0.4       2.0   158.443      10.9   799.522
```

7.10 Сравнительный график всех траекторий I(t)

```
p2 = plot(size=(900, 500))
for params in all_params
    data, _ = produce_or_load(
        datadir(script_name, "scan"),
```

```
        params, run_sir;
        prefix = "sir_scan", tag = false, verbose = false
    )
    plot!(p2, data["t"], data["I"],
      label="β=$(params[:β]), γ=$(params[:γ])", lw=2, alpha=0.8)
end
plot!(p2,
    xlabel="□□□□□, □□□", ylabel="□□□□□□□□□□□□□□ I(t)",
    title="SIR: □□□□□□□ β γ □ □□ □□□□□□□□ □□□□□□□□",
    legend=:topright, grid=true)
savefig(plotsdir(script_name, "sir_parametric_comparison.png"))
```

"/home/dabakharev/work/study/2026-1/2026-1==study-simulation-modeli

7.11 График R0 от параметров

```
p3 = plot(results_df.β, results_df.R0,
    seriestype=:scatter, group=results_df.γ,
    xlabel="β", ylabel="R0 = c·β/γ",
    title="□□□□□□□□□□□ R0 □□ β γ □",
    markersize=8, legend=:topleft, grid=true)
hline!(p3, [1.0], color=:red, linestyle=:dash, label="□□□□□ R0 = 1")
savefig(plotsdir(script_name, "sir_R0_vs_params.png"))
```

"/home/dabakharev/work/study/2026-1/2026-1==study-simulation-modeli

7.12 График R(∞) от параметров

```
p4 = plot(results_df.□, results_df.R_inf ./ (results_df.□ .* 0 .+ 1000)
    seriestype=:scatter, group=results_df.□,
    xlabel="□", ylabel="□□□□ □□□□□□□□□□□□, %",
    title="□□□□□□□□ □□□□ □□□□□□□□□□□□",
    markersize=8, legend=:topleft, grid=true)
savefig(plotsdir(script_name, "sir_R_inf_vs_params.png"))
```

```
"/home/dabakharev/work/study/2026-1/2026-1==study-simulation-modeli
```

7.13 Бенчмаркинг для разных значений β

```
println("\n" * "="^60)
println("□□□□□□□□□□□ □□□ □□□□□□ □")
println("="^60)

benchmark_results = []
for □_value in param_grid[:□]
    bench_params = Dict(
        :u0    => [990.0, 10.0, 0.0],
        :□     => □_value,
        :c     => 10.0,
        :□     => 0.25,
        :tspan => (0.0, 40.0)
    )

    function benchmark_run()
```

```
        prob = ODEProblem(sir_ode!, bench_params[:u0], bench_params[:ts
            (bench_params[:□], bench_params[:c], bench_params[:□]))
            return solve(prob, Tsit5(); saveat=0.1)
        end

        println("\n□□□□□□□□ □□□ □ = $□_value:")
        b = @benchmark $benchmark_run() samples=100 evals=1
    push!(benchmark_results, (□=□_value, time=median(b).time/1e9))
    println("  □□□□□□□ □□□□□: ", round(median(b).time/1e9; digits=4), " □□
end

bench_df = DataFrame(benchmark_results)
p5 = plot(bench_df.□, bench_df.time,
    seriestype=:scatter,
    label="□□□□□ □□□□□□□",
    xlabel="□", ylabel="□□□□□, □□□",
    title="□□□□□□□□□□□ □□□□□□□ □□□□□□□ □□ □",
    markersize=8, markercolor=:green, legend=:topleft, grid=true)
savefig(plotsdir(script_name, "sir_computation_time_vs_beta.png"))
```

```
============================================================
□□□□□□□□□□ □□□ □□□□□□ □
============================================================

□□□□□□□□ □□□ □ = 0.03:
  □□□□□□□ □□□□□: 0.0001 □□□

□□□□□□□□ □□□ □ = 0.05:
```

```
  □□□□□□□ □□□□□: 0.0 □□□

□□□□□□□□ □□□ β = 0.08:
  □□□□□□□ □□□□□: 0.0001 □□□

"/home/dabakharev/work/study/2026-1/2026-1==study-simulation-modeli
```

7.14 Сохранение всех результатов

```
@save datadir(script_name, "all_results.jld2") base_params param_gri
@save datadir(script_name, "all_plots.jld2") p1 p2 p3 p4 p5

println("\n" * "="^60)
println("□□□□□□□□□□□□□□□ □□□□□□□□□□□□ SIR □□□□□□□□□")
println("="^60)
println("\n□□□□□□□□□□ □□□□□□□□□ β:")
println("  • data/$script_name/single/       — □□□□□□□ □□□□□□□□□□□")
println("  • data/$script_name/scan/         — □□□□□□□□□□□□□□□ □□□□□□□□□□□□")
println("  • data/$script_name/all_results.jld2 — □□□□□□□ □□□□□□")
println("  • plots/$script_name/               — □□□ □□□□□□□")
```

```
============================================================
□□□□□□□□□□□□□□□ □□□□□□□□□□□ SIR □□□□□□□□□
============================================================

□□□□□□□□□□ □□□□□□□□□ β:
  • data//single/          — □□□□□□□ □□□□□□□□□□□
  • data//scan/            — □□□□□□□□□□□□□□□ □□□□□□□□□□□□
```

- data//all_results.jld2 — [illegible]
- plots// — [illegible]

8 Параметрическое исследование модели Лотки-Вольтерры

Исследуем влияние скорости размножения жертв α и смертности хищников γ на динамику колебаний популяций.

8.1 Активация проекта и загрузка пакетов

```
import Pkg
Pkg.activate("/home/dabakharev/work/study/2026-1/2026-1==study-simu
using DrWatson
using DifferentialEquations
using DataFrames
using Plots
using Statistics
using FFTW
using JLD2
using BenchmarkTools

ENV["GKSwstype"] = "100"
gr(format=:png)
```

```
script_name = splitext(basename(PROGRAM_FILE))[1]
mkpath(plotsdir(script_name))
mkpath(datadir(script_name))
```

```
  Activating project at `~/work/study/2026-1/2026-1==study-simulation
```

```
"/home/dabakharev/work/study/2026-1/2026-1==study-simulation-modeli
```

8.2 Определение модели Лотки-Вольтерры

dx/dt = □ x - □ x y (□□□□□□)

dy/dt = □ x y - □ y (□□□□□□□)

```
function lotka_volterra!(du, u, p, t)
    x, y = u
    □, □, □, □ = p
    @inbounds begin
        du[1] = □*x - □*x*y
        du[2] = □*x*y - □*y
    end
    nothing
end
```

```
lotka_volterra! (generic function with 1 method)
```

8.3 Базовые параметры эксперимента

```
base_params = Dict(
    :u0    => [[40.0, 9.0]],
    :□     => [0.1],
    :□     => [0.02],
    :□     => [0.01],
    :□     => [0.3],
    :tspan => [(0.0, 200.0)],
    :saveat => [0.1]
)

println("□□□□□□□ □□□□□□□□□ □□□□□□□□□□□□:")
for (key, value) in base_params
    println("  $key = $value")
end
```

```
□□□□□□□ □□□□□□□□□ □□□□□□□□□□□□:
  □ = [0.1]
  □ = [0.3]
  u0 = [[40.0, 9.0]]
  saveat = [0.1]
  □ = [0.01]
  □ = [0.02]
  tspan = [(0.0, 200.0)]
```

8.4 Функция запуска одного эксперимента

```
function run_lv(params)
   prob = ODEProblem(lotka_volterra!, params[:u0], params[:tspan],
        (params[:□], params[:□], params[:□], params[:□]))
    sol = solve(prob, Tsit5();
       reltol=1e-8, abstol=1e-10, saveat=params[:saveat])

    prey     = [u[1] for u in sol.u]
    predator = [u[2] for u in sol.u]

    x_star = params[:□] / params[:□]   # □□□□□□□□□□ □□□□□
    y_star = params[:□] / params[:□]   # □□□□□□□□□□ □□□□□□□□

    return Dict(
        "t"             => sol.t,
        "prey"          => prey,
        "predator"      => predator,
        "prey_mean"     => mean(prey),
        "predator_mean" => mean(predator),
        "prey_max"      => maximum(prey),
        "prey_min"      => minimum(prey),
        "predator_max"  => maximum(predator),
        "predator_min"  => minimum(predator),
        "x_star"        => x_star,
        "y_star"        => y_star
    )
end
```

```
run_lv (generic function with 1 method)
```

8.5 Запуск базового эксперимента

```
base_params_single = dict_list(base_params)[1]
data, path = produce_or_load(
    datadir(script_name, "single"),
    base_params_single,
    run_lv;
    prefix = "lv_base",
    tag = false,
    verbose = true
)

println("\n□□□□□□□□□□ □□□□□□□□ □□□□□□□□□□□□:")
println("  x*        = ", round(data["x_star"], digits=3))
println("  y*        = ", round(data["y_star"], digits=3))
println("  <prey>    = ", round(data["prey_mean"], digits=2))
println("  <pred>    = ", round(data["predator_mean"], digits=2))
println("  □□□□ □□□□□□□□□□: ", path)
```

```
□□□□□□□□□□ □□□□□□□□ □□□□□□□□□□□□:
  x*        = 30.0
  y*        = 5.0
  <prey>    = 29.71
  <pred>    = 5.2
  □□□□ □□□□□□□□□□: /home/dabakharev/work/study/2026-1/2026-1==study-sim
```

8.6 Визуализация базового эксперимента

```
p1 = plot(data["t"], [data["prey"] data["predator"]],
    label=["□□□□□□ x(t)" "□□□□□□□ y(t)"],
    xlabel="□□□□□", ylabel="□□□□□□□□□",
    title="□□□□□-□□□□□□□□□: □□□□□□□ □□□□□□□□□□□",
    lw=2, legend=:topright, grid=true,
    color=[:green :red])
savefig(plotsdir(script_name, "lv_single.png"))
```

```
"/home/dabakharev/work/study/2026-1/2026-1==study-simulation-modeli
```

8.7 Параметрическое сканирование по α и γ

```
param_grid = Dict(
    :u0    => [[40.0, 9.0]],
    :□     => [0.05, 0.1, 0.2],        # □□□□□□□□□□□ □
    :□     => [0.02],                  # □□□□□□□□□
    :□     => [0.01],                  # □□□□□□□□□
    :□     => [0.2, 0.3, 0.4],         # □□□□□□□□□□□ □
    :tspan => [(0.0, 200.0)],
    :saveat => [0.1]
)

all_params = dict_list(param_grid)

println("\n" * "="^60)
println("□□□□□□□□□□□□□□□ □□□□□□□□□□□ □□□□□-□□□□□□□□")
```

```
println("□□□□□ □□□□□□□□□□: ", length(all_params))
println("□□□□□□□□□□ □: ", param_grid[:□])
println("□□□□□□□□□□ □: ", param_grid[:□])
println("="^60)
```

```
============================================================
□□□□□□□□□□□□□□□ □□□□□□□□□□□ □□□□□-□□□□□□□□□
□□□□□ □□□□□□□□□: 9
□□□□□□□□□□ □: [0.05, 0.1, 0.2]
□□□□□□□□□□ □: [0.2, 0.3, 0.4]
============================================================
```

8.8 Запуск всех экспериментов и сбор сводных результатов

```
all_results = []
all_dfs = []

for (i, params) in enumerate(all_params)
   println("□□□□□□□□: $i/$(length(all_params)) | □=$(params[:□]), □=$(pa
    data, path = produce_or_load(
        datadir(script_name, "scan"),
        params,
        run_lv;
        prefix = "lv_scan",
        tag = false,
        verbose = false
```

```
    )
    result_summary = merge(params, Dict(
        :x_star        => data["x_star"],
        :y_star        => data["y_star"],
        :prey_mean     => data["prey_mean"],
        :predator_mean => data["predator_mean"],
        :prey_max      => data["prey_max"],
        :prey_min      => data["prey_min"],
        :filepath      => path
    ))
    push!(all_results, result_summary)

    df = DataFrame(
        t        = data["t"],
        prey     = data["prey"],
        predator = data["predator"],
        □ = fill(params[:□], length(data["t"])),
        □ = fill(params[:□], length(data["t"]))
    )
    push!(all_dfs, df)
end
```

□□□□□□□□: 1/9 | □=0.05, □=0.2
□□□□□□□□: 2/9 | □=0.1, □=0.2
□□□□□□□□: 3/9 | □=0.2, □=0.2
□□□□□□□□: 4/9 | □=0.05, □=0.3
□□□□□□□□: 5/9 | □=0.1, □=0.3
□□□□□□□□: 6/9 | □=0.2, □=0.3
□□□□□□□□: 7/9 | □=0.05, □=0.4

□□□□□□□□: 8/9 | □=0.1, □=0.4

□□□□□□□□: 9/9 | □=0.2, □=0.4

8.9 Сводная таблица

```
results_df = DataFrame(all_results)
println("\n□□□□□□□ □□□□□□□ □□□□□□□□□□:")
println(results_df[:, [:□, :□, :x_star, :y_star, :prey_mean, :predator
```

□□□□□□□ □□□□□□□ □□□□□□□□□□:

9×6 DataFrame

Row	□	□	x_star	y_star	prey_mean	predator_mean
	Float64	Float64	Float64	Float64	Float64	Float64
1	0.05	0.2	20.0	2.5	17.9447	2.83171
2	0.1	0.2	20.0	5.0	20.2292	5.12398
3	0.2	0.2	20.0	10.0	20.1669	10.0049
4	0.05	0.3	30.0	2.5	28.6822	2.73093
5	0.1	0.3	30.0	5.0	29.7052	5.20396
6	0.2	0.3	30.0	10.0	29.8646	10.1004
7	0.05	0.4	40.0	2.5	39.9998	2.49706
8	0.1	0.4	40.0	5.0	39.7093	5.08706
9	0.2	0.4	40.0	10.0	40.0	9.99982

8.10 Сравнительный график жертв

```
p2 = plot(size=(900, 500))
for params in all_params
    data, _ = produce_or_load(
        datadir(script_name, "scan"),
        params, run_lv;
        prefix = "lv_scan", tag = false, verbose = false
    )
    plot!(p2, data["t"], data["prey"],
      label="□=$(params[:□]), □=$(params[:□])", lw=2, alpha=0.8)
end
plot!(p2,
    xlabel="□□□□□", ylabel="□□□□□□ x(t)",
    title="□□□□□-□□□□□□□□□: □□□□□□□ □ □ □ □□ □□□□□",
    legend=:topright, grid=true)
savefig(plotsdir(script_name, "lv_parametric_prey.png"))
```

"/home/dabakharev/work/study/2026-1/2026-1==study-simulation-modeli

8.11 Сравнительный график хищников

```
p3 = plot(size=(900, 500))
for params in all_params
    data, _ = produce_or_load(
        datadir(script_name, "scan"),
        params, run_lv;
        prefix = "lv_scan", tag = false, verbose = false
```

```
    )
    plot!(p3, data["t"], data["predator"],
      label="□=$(params[:□]), □=$(params[:□])", lw=2, alpha=0.8)
end
plot!(p3,
    xlabel="□□□□□", ylabel="□□□□□□□ y(t)",
    title="□□□□□-□□□□□□□□□: □□□□□□□ □ □ □ □□ □□□□□□□□",
    legend=:topright, grid=true)
savefig(plotsdir(script_name, "lv_parametric_predator.png"))
```

```
"/home/dabakharev/work/study/2026-1/2026-1==study-simulation-modeli
```

8.12 Фазовые портреты для всех комбинаций

```
p4 = plot(size=(800, 600))
for params in all_params
    data, _ = produce_or_load(
        datadir(script_name, "scan"),
        params, run_lv;
        prefix = "lv_scan", tag = false, verbose = false
    )
    plot!(p4, data["prey"], data["predator"],
      label="□=$(params[:□]), □=$(params[:□])", lw=1.5, alpha=0.8)
end
plot!(p4,
    xlabel="□□□□□□ x", ylabel="□□□□□□□ y",
    title="□□□□□□□ □□□□□□□□ □□□ □□□□□□ □ □ □",
    legend=:topright, grid=true)
```

```
savefig(plotsdir(script_name, "lv_parametric_phase.png"))
```

```
"/home/dabakharev/work/study/2026-1/2026-1==study-simulation-modeli
```

8.13 Спектральный анализ — период колебаний

```
function compute_dominant_period(signal, dt)
    n = length(signal)
    spectrum = abs.(rfft(signal .- mean(signal)))
    freq = rfftfreq(n, 1/dt)
    if length(spectrum) > 1
        idx = argmax(spectrum[2:end]) + 1
        return 1 / freq[idx]
    else
        return NaN
    end
end

periods = []
for params in all_params
    data, _ = produce_or_load(
        datadir(script_name, "scan"),
        params, run_lv;
        prefix = "lv_scan", tag = false, verbose = false
    )
    T = compute_dominant_period(data["prey"], 0.1)
    push!(periods, (□=params[:□], □=params[:□], period=T))
end
```

```
period_df = DataFrame(periods)
println("\n□□□□□□ □□□□□□□□□ □□□□□ □□□ □□□□□□ □ □ □:")
println(period_df)

p5 = plot(period_df.□, period_df.period,
    seriestype=:scatter, group=period_df.□,
    xlabel="□", ylabel="□□□□□□ □□□□□□□□ T",
    title="□□□□□□ □□□□□□□□ □□□□□ □□ □ □ □",
    markersize=8, legend=:topright, grid=true)
savefig(plotsdir(script_name, "lv_period_vs_params.png"))
```

```
□□□□□□ □□□□□□□□ □□□□□ □□□ □□□□□□ □ □ □:
9×3 DataFrame
 Row □ □        □        period
     □ Float64  Float64  Float64
─────┼──────────────────────────
   1 □    0.05       0.2  100.05
   2 □    0.1        0.2   50.025
   3 □    0.2        0.2   33.35
   4 □    0.05       0.3   66.7
   5 □    0.1        0.3   40.02
   6 □    0.2        0.3   25.0125
   7 □    0.05       0.4   50.025
   8 □    0.1        0.4   33.35
   9 □    0.2        0.4   22.2333

"/home/dabakharev/work/study/2026-1/2026-1==study-simulation-modeli
```

8.14 Бенчмаркинг

```
println("\n" * "="^60)
println("□□□□□□□□□□□□ □□□ □□□□□□ □")
println("="^60)

benchmark_results = []
for □_value in param_grid[:□]
    bench_params = Dict(
        :u0    => [40.0, 9.0],
        :□     => □_value,
        :□     => 0.02,
        :□     => 0.01,
        :□     => 0.3,
        :tspan => (0.0, 200.0)
    )

    function benchmark_run()
     prob = ODEProblem(lotka_volterra!, bench_params[:u0], bench_para
        (bench_params[:□], bench_params[:□], bench_params[:□], bench_
     return solve(prob, Tsit5(); reltol=1e-8, abstol=1e-10, saveat=0.
    end

    println("\n□□□□□□□□ □□□ □ = $□_value:")
    b = @benchmark $benchmark_run() samples=50 evals=1
  push!(benchmark_results, (□=□_value, time=median(b).time/1e9))
  println("  □□□□□□□ □□□□□: ", round(median(b).time/1e9; digits=4), " □□
end
```

```
bench_df = DataFrame(benchmark_results)
p6 = plot(bench_df.□, bench_df.time,
    seriestype=:scatter,
    label="□□□□□ □□□□□□□",
    xlabel="□", ylabel="□□□□□, □□□",
    title="□□□□□□□□□□□ □□□□□□□ □□□□□□□ □□ □",
   markersize=8, markercolor=:green, legend=:topleft, grid=true)
savefig(plotsdir(script_name, "lv_computation_time_vs_alpha.png"))
```

```
============================================================
□□□□□□□□□□ □□□ □□□□□□ □
============================================================

□□□□□□□□ □□□ □ = 0.05:
  □□□□□□□ □□□□□: 0.0003 □□□

□□□□□□□□ □□□ □ = 0.1:
  □□□□□□□ □□□□□: 0.0003 □□□

□□□□□□□□ □□□ □ = 0.2:
  □□□□□□□ □□□□□: 0.0003 □□□

"/home/dabakharev/work/study/2026-1/2026-1==study-simulation-modeli
```

8.15 Сохранение всех результатов

```
@save datadir(script_name, "all_results.jld2") base_params param_gric
@save datadir(script_name, "all_plots.jld2") p1 p2 p3 p4 p5 p6

println("\n" * "="^60)
println("□□□□□□□□□□□□□□□□ □□□□□□□□□□□□ □□□□□-□□□□□□□□□ □□□□□□□□□")
println("="^60)
println("\n□□□□□□□□□□ □□□□□□□□□ □:")
println("  • data/$script_name/single/       — □□□□□□□ □□□□□□□□□□□")
println("  • data/$script_name/scan/         — □□□□□□□□□□□□□□□ □□□□□□□□□□□□")
println("  • data/$script_name/all_results.jld2 — □□□□□□□ □□□□□□")
println("  • plots/$script_name/               — □□□ □□□□□□□")
```

```
============================================================
□□□□□□□□□□□□□□□□ □□□□□□□□□□□□ □□□□□-□□□□□□□□□ □□□□□□□□□
============================================================

□□□□□□□□□□ □□□□□□□□□ □:
  • data//single/          — □□□□□□□ □□□□□□□□□□□
  • data//scan/            — □□□□□□□□□□□□□□□ □□□□□□□□□□□□
  • data//all_results.jld2 — □□□□□□□ □□□□□□
  • plots//                — □□□ □□□□□□□
```

9 Заключение

В ходе лабораторной работы №2:

- Реализованы две базовые модели имитационного моделирования: SIR и Лотки-Вольтерра;
- Освоены численные методы решения ОДУ в Julia с использованием DrWatson и DifferentialEquations;
- Проведены параметрические исследования обеих моделей;
- Проанализированы графики динамики и фазовые портреты;
- Отчёт собран в PDF и DOCX, презентация — в PDF/HTML/Markdown.

Список литературы